

Verification Appendix(Testbench)

DOCUMENT OVERVIEW

This appendix contains the complete SystemVerilog testbenches used for unit-level verification of all RTL modules of the 5-Stage Pipelined RISC-V Processor.

Each testbench includes:

- DUT Instantiation
- Functional Coverage
- Assertions
- Directed Testcases
- PASS/FAIL Verification Messages
- Coverage Reporting

Total Modules Verified : 24

Verification Methodology : Directed Verification +
Assertions + Functional Coverage

Table of Contents

1. ALU	3
2. ALU Contol.....	20
3. ADDER	32
4. Branch Decision.....	37
5. Control.....	45
6. Control Mux	53
7. Data Memory	59
8. Ex_Mem	74
9. Forwarding	87
10. Hazard Detection.....	95
11. ID_EX	102
12. IF_ID	119
13. imm_gen	130
14. Instruction Memory	139
15. Mem_wb	145
16. Mux_wb	155
17. Mux	161
18. MuxSrc1	167
19. MuxSrc2	173
20. MuxSrcImm	179
21. PC	184
22. Pc_offset.....	191
23. Regfile	197
24. Risc_top.....	212
25. Run.do	225

1.ALU

```
// Code your testbench here
```

```
// or browse Examples
```

```
//`include "defines.vh"
```

```
`timescale 1ns/1ps
```

```
module tb_alu;
```

```
reg [31:0] src1;
```

```
reg [31:0] src2;
```

```
reg [5:0] operation;
```

```
wire [31:0] alu_result;
```

```
wire branch_condn;
```

```
// DUT
```

```
alu dut(
```

```
    .src1(src1),
```

```
    .src2(src2),
```

```
    .operation(operation),
```

```
    .alu_result(alu_result),
```

```
    .branch_condn(branch_condn)
```

```
);
```

```
////////////////////////////////////
```

```
// COVERAGE
```

```
////////////////////////////////////
```

```
covergroup alu_cg @(operation);
```

```
    option.per_instance = 1;
```

```
option.name = "ALU_COVERAGE";
```

```
cp_operation : coverpoint operation {
```

```
    bins add_bin = {`ALU_ADD};
```

```
    bins sub_bin = {`ALU_SUB};
```

```
    bins sll_bin = {`ALU_SLL};
```

```
    bins srl_bin = {`ALU_SRL};
```

```
    bins sra_bin = {`ALU_SRA};
```

```
    bins slt_bin = {`ALU_SLT};
```

```
    bins sltu_bin = {`ALU_SLTU};
```

```
    bins xor_bin = {`ALU_XOR};
```

```
    bins or_bin = {`ALU_OR};
```

```
    bins and_bin = {`ALU_AND};
```

```
    bins addi_bin = {`ALU_ADDI};
```

```
    bins loadstore_bin = {
```

```
        `ALU_L_BYTE,
```

```
        `ALU_L_HALF,
```

```
        `ALU_L_WORD,
```

```
        `ALU_L_BU,
```

```
        `ALU_L_HU,
```

```
        `ALU_S_BYTE,
```

```
        `ALU_S_HALF,
```

```
        `ALU_S_WORD
```

```
    };
```

```
    bins beq_bin = {`ALU_BEQ};
```

```
    bins bne_bin = {`ALU_BNE};
```

```
    bins blt_bin = {`ALU_BLT};
```

```

    bins bge_bin = {`ALU_BGE};
    bins bltu_bin = {`ALU_BLTU};
    bins bgeu_bin = {`ALU_BGEU};

    bins lui_bin = {`ALU_LUI};
    bins jal_bin = {`ALU_JAL};

    bins default_bin = default;
}

endgroup

alu_cg cg = new();

////////////////////////////////////
// TESTCASES
////////////////////////////////////

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_alu);

    //-----
    // TC1 ADD
    //-----

    $display("\nTC1 : ADD");

    src1 = 10;
    src2 = 20;
    operation = `ALU_ADD;

    #5;

```

```

if(alu_result == 30)
    $display("PASS : ADD");
else
    $display("FAIL : ADD");

if(alu_result == (src1 + src2))
    $display("ASSERT PASS : ADD");
else
    $error("ASSERT FAIL : ADD");

//-----
// TC2 SUB
//-----

$display("\nTC2 : SUB");

src1 = 50;
src2 = 20;
operation = `ALU_SUB;

#5;

if(alu_result == 30)
    $display("PASS : SUB");
else
    $display("FAIL : SUB");

if(alu_result == (src1 - src2))
    $display("ASSERT PASS : SUB");
else
    $error("ASSERT FAIL : SUB");

//-----

```

```
// TC3 AND
```

```
//-----
```

```
$display("\nTC3 : AND");
```

```
src1 = 32'hF0F0;
```

```
src2 = 32'h0FF0;
```

```
operation = `ALU_AND;
```

```
#5;
```

```
if(alu_result == 32'h00F0)
```

```
    $display("PASS : AND");
```

```
else
```

```
    $display("FAIL : AND");
```

```
if(alu_result == (src1 & src2))
```

```
    $display("ASSERT PASS : AND");
```

```
else
```

```
    $error("ASSERT FAIL : AND");
```

```
//-----
```

```
// TC4 OR
```

```
//-----
```

```
$display("\nTC4 : OR");
```

```
operation = `ALU_OR;
```

```
#5;
```

```
if(alu_result == 32'hFFF0)
```

```
    $display("PASS : OR");
```

```
else
```

```

        $display("FAIL : OR");

    if(alu_result == (src1 | src2))
        $display("ASSERT PASS : OR");
    else
        $error("ASSERT FAIL : OR");

    //-----
    // TC5 XOR
    //-----

    $display("\nTC5 : XOR");

    operation = `ALU_XOR;

    #5;

    if(alu_result == 32'hFF00)
        $display("PASS : XOR");
    else
        $display("FAIL : XOR");

    if(alu_result == (src1 ^ src2))
        $display("ASSERT PASS : XOR");
    else
        $error("ASSERT FAIL : XOR");

    //-----
    // TC6 SLL
    //-----

    $display("\nTC6 : SLL");

    src1 = 1;

```



```

src2 = 4;
operation = `ALU_SLL;

#5;

if(alu_result == 16)
    $display("PASS : SLL");
else
    $display("FAIL : SLL");

if(alu_result == (src1 << src2[4:0]))
    $display("ASSERT PASS : SLL");
else
    $error("ASSERT FAIL : SLL");

//-----
// TC7 SRL
//-----

$display("\nTC7 : SRL");

src1 = 32;
src2 = 2;
operation = `ALU_SRL;

#5;

if(alu_result == 8)
    $display("PASS : SRL");
else
    $display("FAIL : SRL");

if(alu_result == (src1 >> src2[4:0]))
    $display("ASSERT PASS : SRL");

```

```

else
    $error("ASSERT FAIL : SRL");

    //-----
    // TC8 SRA
    //-----

    $display("\nTC8 : SRA");

    src1 = -32;
    src2 = 2;
    operation = `ALU_SRA;

    #5;

    if($signed(alu_result) == -8)
        $display("PASS : SRA");
    else
        $display("FAIL : SRA");

    if($signed(alu_result) == ($signed(src1) >>> src2[4:0]))
        $display("ASSERT PASS : SRA");
else
    $error("ASSERT FAIL : SRA");

    //-----
    // TC9 SLT
    //-----

    $display("\nTC9 : SLT");

    src1 = -5;
    src2 = 10;
    operation = `ALU_SLT;

```

```
#5;
```

```
if(alu_result == 1)
```

```
    $display("PASS : SLT");
```

```
else
```

```
    $display("FAIL : SLT");
```

```
if(alu_result == (($signed(src1) < $signed(src2)) ? 1 : 0))
```

```
    $display("ASSERT PASS : SLT");
```

```
else
```

```
    $error("ASSERT FAIL : SLT");
```

```
//-----
```

```
// TC10 SLTU
```

```
//-----
```

```
$display("\nTC10 : SLTU");
```

```
src1 = 5;
```

```
src2 = 10;
```

```
operation = `ALU_SLTU;
```

```
#5;
```

```
if(alu_result == 1)
```

```
    $display("PASS : SLTU");
```

```
else
```

```
    $display("FAIL : SLTU");
```

```
if(alu_result == ((src1 < src2) ? 1 : 0))
```

```
    $display("ASSERT PASS : SLTU");
```

```
else
```

```
    $error("ASSERT FAIL : SLTU");
```

```
//-----  
// TC11 LOAD ADDRESS  
//-----
```

```
$display("\nTC11 : LOAD");
```

```
src1 = 100;
```

```
src2 = 20;
```

```
operation = `ALU_L_WORD;
```

```
#5;
```

```
if(alu_result == 120)
```

```
    $display("PASS : LOAD");
```

```
else
```

```
    $display("FAIL : LOAD");
```

```
if(alu_result == (src1 + src2))
```

```
    $display("ASSERT PASS : LOAD");
```

```
else
```

```
    $error("ASSERT FAIL : LOAD");
```

```
//-----  
// TC12 BEQ  
//-----
```

```
$display("\nTC12 : BEQ");
```

```
src1 = 10;
```

```
src2 = 10;
```

```
operation = `ALU_BEQ;
```

```
#5;
```

```
if(branch_condn)
```

```
    $display("PASS : BEQ");
```

```
else
```

```
    $display("FAIL : BEQ");
```

```
if(branch_condn == (src1 == src2))
```

```
    $display("ASSERT PASS : BEQ");
```

```
else
```

```
    $error("ASSERT FAIL : BEQ");
```

```
//-----
```

```
// TC13 BNE
```

```
//-----
```

```
$display("\nTC13 : BNE");
```

```
src1 = 10;
```

```
src2 = 20;
```

```
operation = `ALU_BNE;
```

```
#5;
```

```
if(branch_condn)
```

```
    $display("PASS : BNE");
```

```
else
```

```
    $display("FAIL : BNE");
```

```
if(branch_condn == (src1 != src2))
```

```
    $display("ASSERT PASS : BNE");
```

```
else
```

```
    $error("ASSERT FAIL : BNE");
```

```
//-----  
// TC14 BLT  
//-----
```

```
$display("\nTC14 : BLT");
```

```
src1 = 5;
```

```
src2 = 10;
```

```
operation = `ALU_BLT;
```

```
#5;
```

```
if(branch_condn)
```

```
    $display("PASS : BLT");
```

```
else
```

```
    $display("FAIL : BLT");
```

```
if(branch_condn == ($signed(src1) < $signed(src2)))
```

```
    $display("ASSERT PASS : BLT");
```

```
else
```

```
    $error("ASSERT FAIL : BLT");
```

```
//-----  
// TC15 BGE  
//-----
```

```
$display("\nTC15 : BGE");
```

```
src1 = 10;
```

```
src2 = 5;
```

```
operation = `ALU_BGE;
```

```
#5;
```

```

if(branch_condn)
    $display("PASS : BGE");
else
    $display("FAIL : BGE");

if(branch_condn == ($signed(src1) >= $signed(src2)))
    $display("ASSERT PASS : BGE");
else
    $error("ASSERT FAIL : BGE");

//-----
// TC16 LUI
//-----

$display("\nTC16 : LUI");

src2 = 32'h12345000;
operation = `ALU_LUI;

#5;

if(alu_result == 32'h12345000)
    $display("PASS : LUI");
else
    $display("FAIL : LUI");

if(alu_result == src2)
    $display("ASSERT PASS : LUI");
else
    $error("ASSERT FAIL : LUI");

//-----
// TC17 JAL

```

```

//-----

$display("\nTC17 : JAL");

operation = `ALU_JAL;

#5;

if(alu_result == 0)
    $display("PASS : JAL");
else
    $display("FAIL : JAL");

if(alu_result == 0)
    $display("ASSERT PASS : JAL");
else
    $error("ASSERT FAIL : JAL");

//-----
// TC18 DEFAULT
//-----

$display("\nTC18 : DEFAULT");

operation = 6'b111111;

#5;

if(alu_result == 32'hDEADBEEF)
    $display("PASS : DEFAULT");
else
    $display("FAIL : DEFAULT");

if(alu_result == 32'hDEADBEEF)

```



```

    $display("ASSERT PASS : DEFAULT");
else
    $error("ASSERT FAIL : DEFAULT");

//-----
// TC19 : ADDI
//-----

$display("\nTC19 : ADDI");

src1 = 50;
src2 = 25;
operation = `ALU_ADDI;

#5;

if(alu_result == 75)
    $display("PASS : ADDI");
else
    $display("FAIL : ADDI");

if(alu_result == ($signed(src1) + $signed(src2)))
    $display("ASSERT PASS : ADDI");
else
    $error("ASSERT FAIL : ADDI");

//-----
// TC20 : BLTU
//-----

$display("\nTC20 : BLTU");

src1 = 5;
src2 = 10;

```

```
operation = `ALU_BLTU;
```

```
#5;
```

```
if(branch_condn)
```

```
    $display("PASS : BLTU");
```

```
else
```

```
    $display("FAIL : BLTU");
```

```
if(branch_condn == (src1 < src2))
```

```
    $display("ASSERT PASS : BLTU");
```

```
else
```

```
    $error("ASSERT FAIL : BLTU");
```

```
//-----
```

```
// TC21 : BGEU
```

```
//-----
```

```
$display("\nTC21 : BGEU");
```

```
src1 = 10;
```

```
src2 = 5;
```

```
operation = `ALU_BGEU;
```

```
#5;
```

```
if(branch_condn)
```

```
    $display("PASS : BGEU");
```

```
else
```

```
    $display("FAIL : BGEU");
```

```
if(branch_condn == (src1 >= src2))
```

```
    $display("ASSERT PASS : BGEU");
```

```
else
```

```
$error("ASSERT FAIL : BGEU");
```

```
//-----
```

```
$display("\nALU VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",  
        cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
$display("\nFinal Coverage = %0.2f%%",  
        cg.get_coverage());
```

```
end
```

```
endmodule
```

2. ALU Contol

```
`timescale 1ns/1ps

module tb_alu_control;

    reg [2:0] alu_op;
    reg [2:0] func3;
    reg [6:0] func7;

    wire [5:0] operation;

    //-----
    // DUT
    //-----

    alu_control dut(
        .alu_op(alu_op),
        .func3(func3),
        .func7(func7),
        .operation(operation)
    );

    //-----
    // COVERAGE
    //-----

    covergroup alu_control_cg @(alu_op);

        option.per_instance = 1;
        option.name = "ALU_CONTROL_COVERAGE";

        cp_aluop : coverpoint alu_op
        {
```

```

    bins load_store = {'ALUOP_LOAD_STORE};
    bins branch     = {'ALUOP_BRANCH};
    bins r_type     = {'ALUOP_R_TYPE};
    bins i_type     = {'ALUOP_I_TYPE};
    bins jal        = {'ALUOP_JAL};
    bins jalr       = {'ALUOP_JALR};
    bins lui        = {'ALUOP_LUI};
    bins auipc      = {'ALUOP_AUIPC};
}

```

```

endgroup

```

```

alu_control_cg cg = new();

```

```

//-----
// TESTCASES
//-----

```

```

initial begin

```

```

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_alu_control);

```

```

//-----
// TC1 LOAD STORE
//-----

```

```

    $display("\nTC1 : LOAD STORE");

```

```

    alu_op = 'ALUOP_LOAD_STORE;
    func3 = 0;
    func7 = 0;

```

```

    #5;

```

```
if(operation == `ALU_ADD)
```

```
    $display("PASS : TC1");
```

```
else
```

```
    $display("FAIL : TC1");
```

```
if(operation == `ALU_ADD)
```

```
    $display("ASSERT PASS : LOAD STORE");
```

```
else
```

```
    $error("ASSERT FAIL : LOAD STORE");
```

```
//-----
```

```
// TC2 BEQ
```

```
//-----
```

```
$display("\nTC2 : BEQ");
```

```
alu_op = `ALUOP_BRANCH;
```

```
func3 = 3'b000;
```

```
#5;
```

```
if(operation == `ALU_BEQ)
```

```
    $display("PASS : TC2");
```

```
else
```

```
    $display("FAIL : TC2");
```

```
if(operation == `ALU_BEQ)
```

```
    $display("ASSERT PASS : BEQ");
```

```
else
```

```
    $error("ASSERT FAIL : BEQ");
```

```
//-----
```

```
// TC3 BNE
```

```
//-----
```

```
$display("\nTC3 : BNE");
```

```
func3 = 3'b001;
```

```
#5;
```

```
if(operation == `ALU_BNE)
```

```
    $display("PASS : TC3");
```

```
else
```

```
    $display("FAIL : TC3");
```

```
if(operation == `ALU_BNE)
```

```
    $display("ASSERT PASS : BNE");
```

```
else
```

```
    $error("ASSERT FAIL : BNE");
```

```
//-----
```

```
// TC4 BLT
```

```
//-----
```

```
$display("\nTC4 : BLT");
```

```
func3 = 3'b100;
```

```
#5;
```

```
if(operation == `ALU_BLT)
```

```
    $display("PASS : TC4");
```

```
else
```

```
    $display("FAIL : TC4");
```

```
if(operation == `ALU_BLT)
```

```

        $display("ASSERT PASS : BLT");
else
    $error("ASSERT FAIL : BLT");

//-----
// TC5 BGE
//-----

$display("\nTC5 : BGE");

func3 = 3'b101;

#5;

if(operation == `ALU_BGE)
    $display("PASS : TC5");
else
    $display("FAIL : TC5");

if(operation == `ALU_BGE)
    $display("ASSERT PASS : BGE");
else
    $error("ASSERT FAIL : BGE");

//-----
// TC6 ADD
//-----

$display("\nTC6 : ADD");

alu_op = `ALUOP_R_TYPE;
func3 = 3'b000;
func7 = 7'b00000000;

```



```
#5;
```

```
if(operation == `ALU_ADD)
```

```
    $display("PASS : TC6");
```

```
else
```

```
    $display("FAIL : TC6");
```

```
if(operation == `ALU_ADD)
```

```
    $display("ASSERT PASS : ADD");
```

```
else
```

```
    $error("ASSERT FAIL : ADD");
```

```
//-----
```

```
// TC7 SUB
```

```
//-----
```

```
$display("\nTC7 : SUB");
```

```
func7 = 7'b0100000;
```

```
#5;
```

```
if(operation == `ALU_SUB)
```

```
    $display("PASS : TC7");
```

```
else
```

```
    $display("FAIL : TC7");
```

```
if(operation == `ALU_SUB)
```

```
    $display("ASSERT PASS : SUB");
```

```
else
```

```
    $error("ASSERT FAIL : SUB");
```

```
//-----
```

```
// TC8 SLL
```

```
//-----
```

```
$display("\nTC8 : SLL");
```

```
func3 = 3'b001;
```

```
#5;
```

```
if(operation == `ALU_SLL)
```

```
    $display("PASS : TC8");
```

```
else
```

```
    $display("FAIL : TC8");
```

```
if(operation == `ALU_SLL)
```

```
    $display("ASSERT PASS : SLL");
```

```
else
```

```
    $error("ASSERT FAIL : SLL");
```

```
//-----
```

```
// TC9 XOR
```

```
//-----
```

```
$display("\nTC9 : XOR");
```

```
func3 = 3'b100;
```

```
#5;
```

```
if(operation == `ALU_XOR)
```

```
    $display("PASS : TC9");
```

```
else
```

```
    $display("FAIL : TC9");
```

```
if(operation == `ALU_XOR)
```

```

        $display("ASSERT PASS : XOR");
else
    $error("ASSERT FAIL : XOR");

//-----
// TC10 SRA
//-----

$display("\nTC10 : SRA");

func3 = 3'b101;
func7 = 7'b0100000;

#5;

if(operation == `ALU_SRA)
    $display("PASS : TC10");
else
    $display("FAIL : TC10");

if(operation == `ALU_SRA)
    $display("ASSERT PASS : SRA");
else
    $error("ASSERT FAIL : SRA");

//-----
// TC11 ADDI
//-----

$display("\nTC11 : ADDI");

alu_op = `ALUOP_I_TYPE;
func3 = 3'b000;

```

```
#5;
```

```
if(operation == `ALU_ADDI)
```

```
    $display("PASS : TC11");
```

```
else
```

```
    $display("FAIL : TC11");
```

```
if(operation == `ALU_ADDI)
```

```
    $display("ASSERT PASS : ADDI");
```

```
else
```

```
    $error("ASSERT FAIL : ADDI");
```

```
//-----
```

```
// TC12 SLLI
```

```
//-----
```

```
$display("\nTC12 : SLLI");
```

```
func3 = 3'b001;
```

```
#5;
```

```
if(operation == `ALU_SLLI)
```

```
    $display("PASS : TC12");
```

```
else
```

```
    $display("FAIL : TC12");
```

```
if(operation == `ALU_SLLI)
```

```
    $display("ASSERT PASS : SLLI");
```

```
else
```

```
    $error("ASSERT FAIL : SLLI");
```

```
//-----
```

```
// TC13 SRLI
```

```
//-----
```

```
$display("\nTC13 : SRLI");
```

```
func3 = 3'b101;
```

```
func7 = 7'b00000000;
```

```
#5;
```

```
if(operation == `ALU_SRLI)
```

```
    $display("PASS : TC13");
```

```
else
```

```
    $display("FAIL : TC13");
```

```
if(operation == `ALU_SRLI)
```

```
    $display("ASSERT PASS : SRLI");
```

```
else
```

```
    $error("ASSERT FAIL : SRLI");
```

```
//-----
```

```
// TC14 JAL
```

```
//-----
```

```
$display("\nTC14 : JAL");
```

```
alu_op = `ALUOP_JAL;
```

```
#5;
```

```
if(operation == `ALU_ADD)
```

```
    $display("PASS : TC14");
```

```
else
```

```
    $display("FAIL : TC14");
```

```

if(operation == `ALU_ADD)
    $display("ASSERT PASS : JAL");
else
    $error("ASSERT FAIL : JAL");

//-----
// TC15 JALR
//-----

$display("\nTC15 : JALR");

alu_op = `ALUOP_JALR;

#5;

if(operation == `ALU_ADD)
    $display("PASS : TC15");
else
    $display("FAIL : TC15");

if(operation == `ALU_ADD)
    $display("ASSERT PASS : JALR");
else
    $error("ASSERT FAIL : JALR");

//-----
// TC16 LUI
//-----

$display("\nTC16 : LUI");
alu_op = `ALUOP_LUI;

#5;

if(operation == `ALU_LUI)
    $display("PASS : TC16");
else

```

```

    $display("FAIL : TC16");

if(operation == `ALU_LUI)
    $display("ASSERT PASS : LUI");
else
    $error("ASSERT FAIL : LUI");
//-----
// TC17 AUIPC
//-----

$display("\nTC17 : AUIPC");
alu_op = `ALUOP_AUIPC;
#5;
if(operation == `ALU_ADD)
    $display("PASS : TC17");
else
    $display("FAIL : TC17");
if(operation == `ALU_ADD)
    $display("ASSERT PASS : AUIPC");
else
    $error("ASSERT FAIL : AUIPC");
//-----

$display("\nALU_CONTROL VERIFICATION COMPLETE");

$display("\nCoverage = %0.2f%%",
    cg.get_coverage());

#10;
$finish;
end
final begin
    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());
end
endmodule

```

3. ADDER

```
`timescale 1ns/1ps
```

```
module tb_adder;
```

```
reg [31:0] pc;
```

```
wire [31:0] pc_plus_4;
```

```
// DUT
```

```
adder dut(  
    .pc(pc),  
    .pc_plus_4(pc_plus_4)  
);
```

```
//-----
```

```
// COVERAGE
```

```
//-----
```

```
covergroup adder_cg @(pc);
```

```
    option.per_instance = 1;
```

```
    option.name = "ADDER_COVERAGE";
```

```
    cp_pc : coverpoint pc
```

```
    {
```

```
        bins zero_val = {0};
```

```
        bins small_val = {[1:100]};
```

```
        bins medium_val = {[101:1000]};
```



```

        bins large_val = {[1001:32'hFFFFFFFE]};

        bins max_val = {32'hFFFFFFF};
    }

endgroup

adder_cg cg = new();

//-----
// TESTCASES
//-----

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_adder);

    //-----
    // TC1 : ZERO
    //-----

    $display("\nTC1 : ZERO");

    pc = 0;

    #5;

    if(pc_plus_4 == 4) begin
        $display("PASS : TC1");
        $display("ASSERT PASS : ZERO");
    end
    else begin
        $display("FAIL : TC1");
    end
end

```

```

        $error("ASSERT FAIL : ZERO");
end

//-----
// TC2 : SMALL VALUE
//-----

$display("\nTC2 : SMALL VALUE");

pc = 20;

#5;

if(pc_plus_4 == 24) begin
    $display("PASS : TC2");
    $display("ASSERT PASS : SMALL");
end
else begin
    $display("FAIL : TC2");
    $error("ASSERT FAIL : SMALL");
end

//-----
// TC3 : MEDIUM VALUE
//-----

$display("\nTC3 : MEDIUM VALUE");

pc = 500;

#5;

if(pc_plus_4 == 504) begin
    $display("PASS : TC3");

```

```
    $display("ASSERT PASS : MEDIUM");
end
else begin
    $display("FAIL : TC3");
    $error("ASSERT FAIL : MEDIUM");
end
```

```
//-----
// TC4 : LARGE VALUE
//-----
```

```
$display("\nTC4 : LARGE VALUE");
```

```
pc = 32'd100000;
```

```
#5;
```

```
if(pc_plus_4 == 32'd100004) begin
    $display("PASS : TC4");
    $display("ASSERT PASS : LARGE");
end
else begin
    $display("FAIL : TC4");
    $error("ASSERT FAIL : LARGE");
end
```

```
//-----
// TC5 : MAX VALUE
//-----
$display("\nTC5 : MAX VALUE");
```

```
pc = 32'hFFFFFFFF;
```

```
#5;
```

```
if(pc_plus_4 == 32'h00000003) begin
    $display("PASS : TC5");
```

```
        $display("ASSERT PASS : OVERFLOW");
    end
    else begin
        $display("FAIL : TC5");
        $error("ASSERT FAIL : OVERFLOW");
    end

    //-----

    $display("\nADDER VERIFICATION COMPLETE");

    $display("\nCoverage = %0.2f%%",
        cg.get_coverage());

    #10;
    $finish;

end

final begin
    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());
end

endmodule
```

4. Branch Decision

```
`timescale 1ns/1ps
module tb_branch_decision;

    reg branch_condn_ex;
    reg jump_ex;
    reg branch_ex;

    wire pc_src;
    wire flush;
    // DUT
    branch_decision dut(
        .branch_condn_ex(branch_condn_ex),
        .jump_ex(jump_ex),
        .branch_ex(branch_ex),
        .pc_src(pc_src),
        .flush(flush)
    );
    //-----
    // COVERAGE
    //-----
    covergroup branch_decision_cg @(branch_condn_ex or jump_ex or branch_ex);
        option.per_instance = 1;
        option.name = "BRANCH_DECISION_COVERAGE";

        cp_branch_condn : coverpoint branch_condn_ex {
            bins zero = {0};
            bins one  = {1};
        }

        cp_branch : coverpoint branch_ex {
            bins zero = {0};
```

```

        bins one = {1};
    }

    cp_jump : coverpoint jump_ex {
        bins zero = {0};
        bins one = {1};
    }

    cp_psrc : coverpoint pc_src {
        bins zero = {0};
        bins one = {1};
    }

    cross_all : cross cp_branch_condn,
                    cp_branch,
                    cp_jump;

endgroup

branch_decision_cg cg = new();

//-----
// Assertions using always @(*)
//-----

always @(*) begin
    // Branch Taken
    if(branch_ex && branch_condn_ex && !jump_ex) begin

        if(pc_src == 1'b1 && flush == 1'b1)
            $display("ASSERT PASS : BRANCH TAKEN");
        else
            $error("ASSERT FAIL : BRANCH TAKEN");
    end
end

```

```

// Branch Not Taken
if(branch_ex && !branch_condn_ex && !jump_ex) begin

    if(pc_src == 1'b0 && flush == 1'b0)
        $display("ASSERT PASS : BRANCH NOT TAKEN");
    else
        $error("ASSERT FAIL : BRANCH NOT TAKEN");

end

// Jump
if(jump_ex) begin

    if(pc_src == 1'b1 && flush == 1'b1)
        $display("ASSERT PASS : JUMP");
    else
        $error("ASSERT FAIL : JUMP");

end

// Idle
if(!branch_ex && !jump_ex) begin

    if(pc_src == 1'b0 && flush == 1'b0)
        $display("ASSERT PASS : IDLE");
    else
        $error("ASSERT FAIL : IDLE");

end

end

//-----

```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_branch_decision);
```

```
//-----
```

```
// TC1
```

```
//-----
```

```
    $display("\nTC1");
```

```
    branch_condn_ex = 0;
```

```
    branch_ex      = 0;
```

```
    jump_ex        = 0;
```

```
    #5;
```

```
    if(pc_src == 0 && flush == 0)
```

```
        $display("PASS : TC1");
```

```
    else
```

```
        $display("FAIL : TC1");
```

```
//-----
```

```
// TC2
```

```
//-----
```

```
    $display("\nTC2");
```

```
    branch_condn_ex = 1;
```

```
    branch_ex      = 0;
```



```
jump_ex      = 0;
```

```
#5;
```

```
if(pc_src == 0)
```

```
    $display("PASS : TC2");
```

```
else
```

```
    $display("FAIL : TC2");
```

```
//-----
```

```
// TC3
```

```
//-----
```

```
$display("\nTC3");
```

```
branch_condn_ex = 0;
```

```
branch_ex      = 1;
```

```
jump_ex        = 0;
```

```
#5;
```

```
if(pc_src == 0)
```

```
    $display("PASS : TC3");
```

```
else
```

```
    $display("FAIL : TC3");
```

```
//-----
```

```
// TC4
```

```
//-----
```

```
$display("\nTC4");
```

```
branch_condn_ex = 1;
```

```
branch_ex      = 1;
```

```
jump_ex       = 0;
```

```
#5;
```

```
if(pc_src == 1 && flush == 1)
```

```
    $display("PASS : TC4");
```

```
else
```

```
    $display("FAIL : TC4");
```

```
//-----
```

```
// TC5
```

```
//-----
```

```
$display("\nTC5");
```

```
branch_condn_ex = 0;
```

```
branch_ex      = 0;
```

```
jump_ex       = 1;
```

```
#5;
```

```
if(pc_src == 1)
```

```
    $display("PASS : TC5");
```

```
else
```

```
    $display("FAIL : TC5");
```

```
//-----
```

```
// TC6
```

```
//-----
```

```
$display("\nTC6");
```

```
branch_condn_ex = 1;
```

```
branch_ex      = 0;
```

```
jump_ex        = 1;
```

```
#5;
```

```
if(pc_src == 1)
```

```
    $display("PASS : TC6");
```

```
else
```

```
    $display("FAIL : TC6");
```

```
//-----
```

```
// TC7
```

```
//-----
```

```
$display("\nTC7");
```

```
branch_condn_ex = 0;
```

```
branch_ex      = 1;
```

```
jump_ex        = 1;
```

```
#5;
```

```
if(pc_src == 1)
```

```
    $display("PASS : TC7");
```

```
else
```

```
    $display("FAIL : TC7");
```

```
//-----
```

```
// TC8
```

```
//-----
```

```
$display("\nTC8");
```

```
branch_condn_ex = 1;
```

```
branch_ex      = 1;
```

```
jump_ex        = 1;
```

```
#5;
```

```
if(pc_src == 1)
```

```
    $display("PASS : TC8");
```

```
else
```

```
    $display("FAIL : TC8");
```

```
//-----
```

```
$display("\nBRANCH_DECISION VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
    $display("\nFinal Coverage = %0.2f%%",
```

```
        cg.get_coverage());
```

```
end
```

```
endmodule
```

5. Control

```
//`include "defines.vh"

`timescale 1ns/1ps

module tb_control;

    reg [6:0] opcode;

    wire regWrite;
    wire aluSrc;
    wire [2:0] aluOp;
    wire branch;
    wire memWrite;
    wire memRead;
    wire memToReg;
    wire jump;
    wire [1:0] result_src;

    control dut(
        .opcode(opcode),
        .regWrite(regWrite),
        .aluSrc(aluSrc),
        .aluOp(aluOp),
        .branch(branch),
        .memWrite(memWrite),
        .memRead(memRead),
        .memToReg(memToReg),
        .jump(jump),
        .result_src(result_src)
    );

    //////////////////////////////////
```

```

// COVERAGE

////////////////////////////////////

covergroup control_cg @(opcode);

option.per_instance = 1;
option.name = "CONTROL_COVERAGE";

cp_opcode : coverpoint opcode
{
    bins r_type = {`OPCODE_R_TYPE};
    bins i_type = {`OPCODE_I_TYPE};
    bins load   = {`OPCODE_LOAD};
    bins store  = {`OPCODE_STORE};
    bins branch = {`OPCODE_BRANCH};
    bins jal    = {`OPCODE_JAL};
    bins jalr   = {`OPCODE_JALR};
    bins lui    = {`OPCODE_LUI};
    bins auipc  = {`OPCODE_AUIPC};
    bins invalid = default;
}

endgroup

control_cg cg = new();

////////////////////////////////////

// TESTCASES

////////////////////////////////////

initial begin

$dumpfile("dump.vcd");
$dumpvars(0,tb_control);

```

```
////////////////////////////////////
```

```
// TC1 R_TYPE
```

```
////////////////////////////////////
```

```
$display("\nTC1 : R_TYPE");
```

```
opcode = `OPCODE_R_TYPE;
```

```
#5;
```

```
if(regWrite && !aluSrc &&
```

```
    aluOp==`ALUOP_R_TYPE &&
```

```
    result_src==2'b00)
```

```
begin
```

```
    $display("PASS : TC1");
```

```
    $display("ASSERT PASS : R_TYPE");
```

```
end
```

```
else
```

```
    $error("ASSERT FAIL : R_TYPE");
```

```
////////////////////////////////////
```

```
// TC2 I_TYPE
```

```
////////////////////////////////////
```

```
$display("\nTC2 : I_TYPE");
```

```
opcode = `OPCODE_I_TYPE;
```

```
#5;
```

```
if(regWrite && aluSrc &&
```

```
    aluOp==`ALUOP_I_TYPE)
```

```
begin
```

```
    $display("PASS : TC2");
```

```
    $display("ASSERT PASS : I_TYPE");
```

```

end
else
    $error("ASSERT FAIL : I_TYPE");

    //////////////////////////////////
    // TC3 LOAD
    //////////////////////////////////

    $display("\nTC3 : LOAD");

    opcode = `OPCODE_LOAD;
    #5;

    if(regWrite &&
        memRead &&
        memToReg &&
        result_src==2'b01)
    begin
        $display("PASS : TC3");
        $display("ASSERT PASS : LOAD");
    end
    else
        $error("ASSERT FAIL : LOAD");

        //////////////////////////////////
        // TC4 STORE
        //////////////////////////////////

        $display("\nTC4 : STORE");

        opcode = `OPCODE_STORE;
        #5;

        if(memWrite &&

```



```

        !regWrite)
begin
    $display("PASS : TC4");
    $display("ASSERT PASS : STORE");
end
else
    $error("ASSERT FAIL : STORE");

////////////////////////////////////
// TC5 BRANCH
////////////////////////////////////

$display("\nTC5 : BRANCH");

opcode = `OPCODE_BRANCH;
#5;

if(branch &&
    !regWrite)
begin
    $display("PASS : TC5");
    $display("ASSERT PASS : BRANCH");
end
else
    $error("ASSERT FAIL : BRANCH");

////////////////////////////////////
// TC6 JAL
////////////////////////////////////

$display("\nTC6 : JAL");

opcode = `OPCODE_JAL;
#5;

```

[illegible]

```
opcode = `OPCODE_LUI;
```

```
#5;
```

```
if(regWrite &&
```

```
    aluOp==`ALUOP_LUI)
```

```
begin
```

```
    $display("PASS : TC8");
```

```
    $display("ASSERT PASS : LUI");
```

```
end
```

```
else
```

```
    $error("ASSERT FAIL : LUI");
```

```
////////////////////////////////////
```

```
// TC9 AUIPC
```

```
////////////////////////////////////
```

```
$display("\nTC9 : AUIPC");
```

```
opcode = `OPCODE_AUIPC;
```

```
#5;
```

```
if(regWrite &&
```

```
    aluOp==`ALUOP_AUIPC)
```

```
begin
```

```
    $display("PASS : TC9");
```

```
    $display("ASSERT PASS : AUIPC");
```

```
end
```

```
else
```

```
    $error("ASSERT FAIL : AUIPC");
```

```
////////////////////////////////////
```

```
// TC10 INVALID OPCODE
```

```
////////////////////////////////////
```

```

$display("\nTC10 : INVALID");

opcode = 7'b1111111;
#5;
if(!regWrite &&
    !aluSrc &&
    !branch &&
    !memWrite &&
    !memRead &&
    !jump)
begin
    $display("PASS : TC10");
    $display("ASSERT PASS : DEFAULT");
end
else
    $error("ASSERT FAIL : DEFAULT");

////////////////////////////////////

$display("\nCONTROL VERIFICATION COMPLETE");

$display("\nCovrage = %0.2f%%",
    cg.get_coverage());

#10;
$finish;
end
final begin
$display("\nFinal Covrage = %0.2f%%",
    cg.get_coverage());
end

endmodule

```

6.Control Mux

```
`timescale 1ns/1ps
```

```
module tb_control_mux;
```

```
    reg control_mux_sel;
```

```
    reg regWrite;
```

```
    reg aluSrc;
```

```
    reg [2:0] aluOp;
```

```
    reg branch;
```

```
    reg memWrite;
```

```
    reg memRead;
```

```
    reg memToReg;
```

```
    reg jump;
```

```
    reg [1:0] result_src;
```

```
    wire regWrite_mux_out;
```

```
    wire aluSrc_mux_out;
```

```
    wire [2:0] aluOp_mux_out;
```

```
    wire branch_mux_out;
```

```
    wire memWrite_mux_out;
```

```
    wire memRead_mux_out;
```

```
    wire memToReg_mux_out;
```

```
    wire jump_mux_out;
```

```
    wire [1:0] result_src_mux_out;
```

```
    control_mux dut(
```

```
        .control_mux_sel(control_mux_sel),
```

```
        .regWrite(regWrite),
```

```
        .aluSrc(aluSrc),
```

```

.aluOp(aluOp),
.branch(branch),
.memWrite(memWrite),
.memRead(memRead),
.memToReg(memToReg),
.jump(jump),
.result_src(result_src),

.regWrite_mux_out(regWrite_mux_out),
.aluSrc_mux_out(aluSrc_mux_out),
.aluOp_mux_out(aluOp_mux_out),
.branch_mux_out(branch_mux_out),
.memWrite_mux_out(memWrite_mux_out),
.memRead_mux_out(memRead_mux_out),
.memToReg_mux_out(memToReg_mux_out),
.jump_mux_out(jump_mux_out),
.result_src_mux_out(result_src_mux_out)

```

```
);
```

```
////////////////////////////////////
```

```
// COVERAGE
```

```
////////////////////////////////////
```

```
covergroup control_mux_cg @(control_mux_sel);
```

```
option.per_instance = 1;
```

```
option.name = "CONTROL_MUX_COVERAGE";
```

```
cp_sel : coverpoint control_mux_sel
```

```

{
    bins normal_flow = {0};
    bins stall_nop   = {1};
}

```

```
endgroup
```

```
control_mux_cg cg = new();
```

```
////////////////////////////////////
```

```
// TESTCASES
```

```
////////////////////////////////////
```

```
initial begin
```

```
$dumpfile("dump.vcd");
```

```
$dumpvars(0,tb_control_mux);
```

```
////////////////////////////////////
```

```
// Common Inputs
```

```
////////////////////////////////////
```

```
regWrite = 1;
```

```
aluSrc   = 1;
```

```
aluOp    = 3'b101;
```

```
branch   = 1;
```

```
memWrite  = 1;
```

```
memRead   = 1;
```

```
memToReg  = 1;
```

```
jump      = 1;
```

```
result_src = 2'b10;
```

```
////////////////////////////////////
```

```
// TC1 : NORMAL FLOW
```

```
////////////////////////////////////
```

```
$display("\nTC1 : NORMAL FLOW");
```

```

control_mux_sel = 0;

#5;

if(
    regWrite_mux_out == regWrite &&
    aluSrc_mux_out  == aluSrc &&
    aluOp_mux_out   == aluOp &&
    branch_mux_out  == branch &&
    memWrite_mux_out == memWrite &&
    memRead_mux_out == memRead &&
    memToReg_mux_out == memToReg &&
    jump_mux_out    == jump &&
    result_src_mux_out == result_src
)
begin
    $display("PASS : TC1");
    $display("ASSERT PASS : NORMAL FLOW");
end
else
    $error("ASSERT FAIL : NORMAL FLOW");

////////////////////////////////////
// TC2 : STALL / NOP INSERTION
////////////////////////////////////

$display("\nTC2 : STALL NOP");

control_mux_sel = 1;

#5;

if(
    regWrite_mux_out == 0 &&

```



```

aluSrc_mux_out == 0 &&
aluOp_mux_out == 0 &&
branch_mux_out == 0 &&
memWrite_mux_out == 0 &&
memRead_mux_out == 0 &&
memToReg_mux_out == 0 &&
jump_mux_out == 0 &&
result_src_mux_out == 0
)
begin
    $display("PASS : TC2");
    $display("ASSERT PASS : STALL NOP");
end
else
    $error("ASSERT FAIL : STALL NOP");

//////////
// TC3 : DYNAMIC SWITCHING
//////////

$display("\nTC3 : DYNAMIC SWITCHING");

control_mux_sel = 0;
#5;

if(regWrite_mux_out == 1)
    $display("ASSERT PASS : NORMAL");

control_mux_sel = 1;
#5;

if(regWrite_mux_out == 0)
    $display("ASSERT PASS : STALL");

```

```
$display("PASS : TC3");
```

```
////////////////////////////////////
```

```
$display("\nCONTROL_MUX VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",  
        cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
$display("\nFinal Coverage = %0.2f%%",  
        cg.get_coverage());
```

```
end
```

```
endmodule
```

7. Data Memory

```
// Code your testbench here
```

```
// or browse Examples
```

```
// Code your testbench here
```

```
// or browse Examples
```

```
`timescale 1ns/1ps
```

```
module tb_data_memory;
```

```
    reg clk;
```

```
    reg rst;
```

```
    reg [31:0] addr;
```

```
    reg [31:0] w_data;
```

```
    reg mem_read;
```

```
    reg mem_write;
```

```
    wire [31:0] r_data;
```

```
// DUT
```

```
data_memory dut(
```

```
    .clk(clk),
```

```
    .rst(rst),
```

```
    .addr(addr),
```

```
    .w_data(w_data),
```

```
    .mem_read(mem_read),
```

```
    .mem_write(mem_write),
```

```
    .r_data(r_data)
```

```
);
```

```
//-----
```

```
// Functional Coverage
```

```
//-----
```

```
covergroup mem_cg @(posedge clk);
```

```
option.per_instance = 1;
```

```
// Reset coverage
```

```
cp_rst : coverpoint rst {  
    bins asserted  = {1};  
    bins deasserted = {0};  
}
```

```
// Read enable coverage
```

```
cp_read : coverpoint mem_read {  
    bins off = {0};  
    bins on  = {1};  
}
```

```
// Write enable coverage
```

```
cp_write : coverpoint mem_write {  
    bins off = {0};  
    bins on  = {1};  
}
```

```
// Address coverage
```

```
cp_addr : coverpoint addr[13:2] {  
  
    bins low_addr = {[0:15]};  
  
    bins mid_addr = {[16:1023]};  
  
    bins high_addr = {[1024:4094]};  
  
    bins boundary = {4095};  
}
```

```

//Data coverage
cp_data : coverpoint w_data {

    bins low_data = {[0:255]};

    bins mid_data = {[256:1023]};

    bins high_data = {[1024:$]};

}

cross_rw : cross cp_read, cp_write;

cross_addr_write : cross cp_addr, cp_write;

endgroup

mem_cg cg = new();

// Clock generation
always #5 clk = ~clk;

//Assertions

property p_read_disable;

@(posedge clk)

(!mem_read) |-> (r_data == 32'b0);

endproperty

```

```
a_read_disable:
assert property(p_read_disable)
else
$error("READ DISABLE FAILED");
```

```
property p_write;
```

```
@(posedge clk)
(mem_write && !rst)
```

```
|=>
```

```
(dut.data_mem[$past(addr[13:2])] == $past(w_data));
```

```
endproperty
```

```
a_write:
assert property(p_write)
else
$error("WRITE FAILED");
```

```
property p_reset;
```

```
@(posedge clk)
```

```
rst |=> (dut.data_mem[0] == 0);
```

```
endproperty
```

```
a_reset:
```

```
assert property(p_reset)
else
$error("RESET FAILED");
```

```
property p_write_read;
```

```
@(posedge clk)
```

```
(mem_write && !rst)
```

```
==>
```

```
##1
```

```
(mem_read)
```

```
|->
```

```
(r_data == dut.data_mem[addr[13:2]]);
```

```
endproperty
```

```
a_write_read:
```

```
assert property(p_write_read)
```

```
else
```

```
$error("READ AFTER WRITE FAILED");
```

```
property p_no_unknown;
```

```
@(posedge clk)
```

```
!$isunknown(r_data);
```

```
endproperty
```

```
a_no_unknown:
```

```
assert property(p_no_unknown)
```

```
else
```

```
$error("UNKNOWN DETECTED");
```

```
initial begin
```

```
    $display("Starting Data Memory Verification");
```

```
    clk = 0;
```

```
    rst = 1;
```

```
    addr = 0;
```

```
    w_data = 0;
```

```
    mem_read = 0;
```

```
    mem_write = 0;
```

```
    // Reset
```

```
    #10;
```

```
    rst = 0;
```

```
    // Write Test
```

```
    addr = 32'h00000004;
```

```
    w_data = 32'd100;
```

```
    mem_write = 1;
```

```
    #10;
```

```
    mem_write = 0;
```

```
    // Read Test
```



```

mem_read = 1;

#5;

if(r_data == 32'd100)
    $display("PASS : Read/Write Test");
else
    $display("FAIL : Read/Write Test");

#20;

//-----
// TC2 : Reset Clears Memory
//-----

$display("\nTC2 : Reset Verification");

// Assert reset
rst = 1;
#10;

rst = 0;

// Read same location
addr = 32'h00000004;
mem_read = 1;

#5;

if(r_data == 32'd0)
    $display("PASS : Reset Cleared Memory");
else
    $display("FAIL : Reset Cleared Memory");

```

```

#10;

//-----
// TC3 : Read Disable Verification
//-----

$display("\nTC3 : Read Disable Verification");

// Write data first

addr = 32'h00000008;
w_data = 32'd200;
mem_write = 1;

#10;

mem_write = 0;

// Disable read

mem_read = 0;

#5;

if(r_data == 32'd0)
    $display("PASS : Read Disable");
else
    $display("FAIL : Read Disable");

#10

//-----
// TC4 : Multiple Address Verification
//-----

```

```
$display("\nTC4 : Multiple Address Verification");
```

```
// Address 4 -> 111
```

```
addr = 32'h00000004;
```

```
w_data = 32'd111;
```

```
mem_write = 1;
```

```
#10;
```

```
// Address 8 -> 222
```

```
addr = 32'h00000008;
```

```
w_data = 32'd222;
```

```
#10;
```

```
// Address 12 -> 333
```

```
addr = 32'h0000000C;
```

```
w_data = 32'd333;
```

```
#10;
```

```
mem_write = 0;
```

```
// Verify Address 4
```

```
addr = 32'h00000004;
```

```
mem_read = 1;
```

```
#5;
```

```

if(r_data == 32'd111)
    $display("PASS : Address 4");
else
    $display("FAIL : Address 4");

// Verify Address 8

addr = 32'h00000008;

#5;

if(r_data == 32'd222)
    $display("PASS : Address 8");
else
    $display("FAIL : Address 8");

// Verify Address 12

addr = 32'h0000000C;

#5;

if(r_data == 32'd333)
    $display("PASS : Address 12");
else
    $display("FAIL : Address 12");

#10

//-----
// TC5 : Boundary Address Verification
//-----

$display("\nTC5 : Boundary Address Verification");

```

```

// Highest legal address

addr = 32'h00003FFC;
w_data = 32'd9999;
mem_write = 1;

#10;

mem_write = 0;

mem_read = 1;

#5;

if(r_data == 32'd9999)
    $display("PASS : Boundary Address");
else
    $display("FAIL : Boundary Address");

#20;

//-----
// TC6 : Mid Address Coverage
//-----

$display("\nTC6 : Mid Address Coverage");

addr = 32'h00001000;
w_data = 32'd555;

mem_write = 1;

#10;

```

```
mem_write = 0;
```

```
mem_read = 1;
```

```
#5;
```

```
if(r_data == 32'd555)
```

```
    $display("PASS : Mid Address");
```

```
else
```

```
    $display("FAIL : Mid Address");
```

```
//-----
```

```
// TC7 : High Address Coverage
```

```
//-----
```

```
$display("\nTC7 : High Address Coverage");
```

```
addr = 32'h00002000;
```

```
w_data = 32'd777;
```

```
mem_write = 1;
```

```
#10;
```

```
mem_write = 0;
```

```
mem_read = 1;
```

```
#5;
```

```
if(r_data == 32'd777)
```

```
    $display("PASS : High Address");
```

```

else
    $display("FAIL : High Address");

    //-----
    // TC8 : Read and Write Simultaneously
    //-----

    $display("\nTC8 : Read + Write Simultaneously");

    addr    = 32'h00000010;
    w_data   = 32'd1234;

    mem_read = 1;
    mem_write = 1;

    #10;

    mem_read = 0;
    mem_write = 0;

    //-----
    // TC9 : Cross Coverage Completion
    //-----

    $display("\nTC9 : Cross Coverage Completion");

    mem_write = 0;
    mem_read  = 0;

    // mid address with write=0
    addr = 32'h00001000;
    #10;

```

```
// high address with write=0
```

```
addr = 32'h00002000;
```

```
#10;
```

```
// boundary address with write=0
```

```
addr = 32'h00003FFC;
```

```
#10;
```

```
//TC10
```

```
$display("\nTC10 : Low Address + Write OFF");
```

```
mem_write = 0;
```

```
mem_read = 0;
```

```
addr = 32'h00000004;
```

```
#10;
```

```
//-----
```

```
// TC11 : Mid Address Bin Completion
```

```
//-----
```

```
$display("\nTC11 : Mid Address Bin Completion");
```

```
addr = 32'h00000100; // addr[13:2] = 64
```

```
w_data = 32'd444;
```

```
mem_write = 1;
```

```
#10;
```

```
mem_write = 0;
```



```
mem_read = 1;
```

```
#5;
```

```
if(r_data == 32'd444)
```

```
    $display("PASS : Mid Address Bin");
```

```
else
```

```
    $display("FAIL : Mid Address Bin");
```

```
$display("\nCoverage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
$finish;
```

```
end
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0, tb_data_memory);
```

```
end
```

```
final begin
```

```
    $display("\nFinal Coverage = %0.2f%%", cg.get_coverage());
```

```
end
```

```
endmodule
```

8. Ex_Mem

```
// Code your testbench here
```

```
// or browse Examples
```

```
`timescale 1ns/1ps
```

```
module tb_ex_mem;
```

```
// Inputs
```

```
reg clk;
```

```
reg rst;
```

```
reg [31:0] alu_result_in;
```

```
reg [31:0] write_data_in;
```

```
reg [4:0] rd_in;
```

```
reg mem_read_in;
```

```
reg mem_write_in;
```

```
reg mem_to_reg_in;
```

```
reg reg_write_in;
```

```
reg [31:0] pc_plus_4_ex;
```

```
reg [1:0] result_src_ex;
```

```
// Outputs
```

```
wire [31:0] alu_result_out;
```

```
wire [31:0] write_data_out;
```

```
wire [4:0] rd_out;
```

```
wire mem_read_out;
```

```
wire mem_write_out;
```

```
wire mem_to_reg_out;
```

```
wire reg_write_out;
```

```

wire [31:0] pc_plus_4_mem;
wire [1:0] result_src_mem;

// DUT
ex_mem dut (
    .clk(clk),
    .rst(rst),

    .alu_result_in(alu_result_in),
    .write_data_in(write_data_in),
    .rd_in(rd_in),

    .mem_read_in(mem_read_in),
    .mem_write_in(mem_write_in),
    .mem_to_reg_in(mem_to_reg_in),
    .reg_write_in(reg_write_in),

    .alu_result_out(alu_result_out),
    .write_data_out(write_data_out),
    .rd_out(rd_out),

    .mem_read_out(mem_read_out),
    .mem_write_out(mem_write_out),
    .mem_to_reg_out(mem_to_reg_out),
    .reg_write_out(reg_write_out),

    .pc_plus_4_ex(pc_plus_4_ex),
    .pc_plus_4_mem(pc_plus_4_mem),

    .result_src_ex(result_src_ex),
    .result_src_mem(result_src_mem)
);

//-----

```

```
// Functional Coverage
```

```
//-----
```

```
covergroup ex_mem_cg @(posedge clk);
```

```
    option.per_instance = 1;
```

```
    option.name = "EX_MEM_COVERAGE";
```

```
    // Reset
```

```
    cp_rst : coverpoint rst {
```

```
        bins asserted  = {1};
```

```
        bins deasserted = {0};
```

```
    }
```

```
    // Control Signals
```

```
    cp_mem_read : coverpoint mem_read_in {
```

```
        bins off = {0};
```

```
        bins on  = {1};
```

```
    }
```

```
    cp_mem_write : coverpoint mem_write_in {
```

```
        bins off = {0};
```

```
        bins on  = {1};
```

```
    }
```

```
    cp_mem_to_reg : coverpoint mem_to_reg_in {
```

```
        bins off = {0};
```

```
        bins on  = {1};
```

```
    }
```

```
    cp_reg_write : coverpoint reg_write_in {
```

```
        bins off = {0};
```

```
        bins on  = {1};
```

```
    }
```

```
// Result Source
```

```
cp_result_src : coverpoint result_src_ex {
```

```
    bins alu_path    = {2'b00};
```

```
    bins mem_path    = {2'b01};
```

```
    bins pc4_path     = {2'b10};
```

```
    bins default_path = {2'b11};
```

```
}
```

```
// Destination Register
```

```
cp_rd : coverpoint rd_in {
```

```
    bins x0          = {0};
```

```
    bins low_reg      = {[1:10]};
```

```
    bins mid_reg       = {[11:20]};
```

```
    bins high_reg      = {[21:31]};
```

```
}
```

```
// Cross Coverage
```

```
cross_result_regwrite :
```

```
    cross cp_result_src, cp_reg_write;
```

```
endgroup
```

```
ex_mem_cg cg = new();
```

```
// Clock generation
```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0, tb_ex_mem);
```

```
    clk = 0;
```

```
    alu_result_in = 0;
```

```
    write_data_in = 0;
```

```
    rd_in = 0;
```

```
    mem_read_in = 0;
```

```
    mem_write_in = 0;
```

```
    mem_to_reg_in = 0;
```

```
    reg_write_in = 0;
```

```
    pc_plus_4_ex = 0;
```

```
    result_src_ex = 0;
```

```
    //-----
```

```
    // TC1 : RESET VERIFICATION
```

```
    //-----
```

```
    $display("\nTC1 : Reset Verification");
```

```
    rst = 1;
```

```
    #12;
```

```
    if (
```

```
        alu_result_out == 0 &&
```

```
        write_data_out == 0 &&
```

```
        rd_out == 0 &&
```

```

    mem_read_out == 0 &&
    mem_write_out == 0 &&
    mem_to_reg_out == 0 &&
    reg_write_out == 0 &&
    pc_plus_4_mem == 0 &&
    result_src_mem == 0
)
    $display("PASS : Reset Verification");
else
    $display("FAIL : Reset Verification");

//-----
// TC2 : DATA PATH TRANSFER
//-----

$display("\nTC2 : Data Path Transfer");

rst = 0;

alu_result_in = 32'd100;
write_data_in = 32'd200;
rd_in = 5;

#10;

if (
    alu_result_out == 32'd100 &&
    write_data_out == 32'd200 &&
    rd_out == 5
)
    $display("PASS : Data Path Transfer");
else
    $display("FAIL : Data Path Transfer");

```

```

if(
    alu_result_out == 32'd100 &&
    write_data_out == 32'd200 &&
    rd_out == 5
)
    $display("ASSERT PASS : DATA PATH");
else
    $error("ASSERT FAIL : DATA PATH");

//-----
// TC3 : CONTROL SIGNAL TRANSFER
//-----

$display("\nTC3 : Control Signal Transfer");

mem_read_in = 1;
mem_write_in = 1;
mem_to_reg_in = 1;
reg_write_in = 1;

#10;

if (
    mem_read_out == 1 &&
    mem_write_out == 1 &&
    mem_to_reg_out == 1 &&
    reg_write_out == 1
)
    $display("PASS : Control Signal Transfer");
else
    $display("FAIL : Control Signal Transfer");

if(

```



```
    mem_read_out == 1 &&
    mem_write_out == 1 &&
    mem_to_reg_out == 1 &&
    reg_write_out == 1
)
    $display("ASSERT PASS : CONTROL PATH");
else
    $error("ASSERT FAIL : CONTROL PATH");
```

```
//-----
// TC4 : PC+4 TRANSFER
//-----
```

```
$display("\nTC4 : PC+4 Transfer");
```

```
pc_plus_4_ex = 32'h00000024;
```

```
#10;
```

```
if (pc_plus_4_mem == 32'h00000024)
    $display("PASS : PC+4 Transfer");
else
    $display("FAIL : PC+4 Transfer");
```

```
if (pc_plus_4_mem == 32'h24)
    $display("ASSERT PASS : PC+4");
else
    $error("ASSERT FAIL : PC+4");
```

```
//-----
// TC5 : RESULT_SRC TRANSFER
//-----
```

```

$display("\nTC5 : result_src Transfer");

result_src_ex = 2'b10;

#10;

if (result_src_mem == 2'b10)
    $display("PASS : result_src Transfer");
else
    $display("FAIL : result_src Transfer");

if(result_src_mem == 2'b10)
    $display("ASSERT PASS : RESULT_SRC");
else
    $error("ASSERT FAIL : RESULT_SRC");

//-----

//-----

// TC6 : result_src = MEM
//-----

$display("\nTC6 : result_src = MEM");

result_src_ex = 2'b01;

#10;

if(result_src_mem == 2'b01)
    $display("ASSERT PASS : RESULT_SRC_MEM");
else
    $error("ASSERT FAIL : RESULT_SRC_MEM");

```

```

//-----
// TC7 : result_src = DEFAULT
//-----

$display("\nTC7 : result_src = DEFAULT");

result_src_ex = 2'b11;

#10;

if(result_src_mem == 2'b11)
    $display("ASSERT PASS : RESULT_SRC_DEFAULT");
else
    $error("ASSERT FAIL : RESULT_SRC_DEFAULT");

//-----
// TC8 : Mid Register
//-----

$display("\nTC8 : Mid Register");

rd_in = 15;

#10;

if(rd_out == 15)
    $display("ASSERT PASS : MID_REGISTER");
else
    $error("ASSERT FAIL : MID_REGISTER");

//-----
// TC9 : High Register
//-----

```

```

$display("\nTC9 : High Register");

rd_in = 25;

#10;

if(rd_out == 25)
    $display("ASSERT PASS : HIGH_REGISTER");
else
    $error("ASSERT FAIL : HIGH_REGISTER");

//-----
// TC10 : mem_path + reg_write ON
//-----

result_src_ex = 2'b01;
reg_write_in = 1;

#10;

if(result_src_mem == 2'b01 && reg_write_out == 1'b1)
    $display("ASSERT PASS : MEM_PATH_REGWRITE_ON");
else
    $error("ASSERT FAIL : MEM_PATH_REGWRITE_ON");

//-----
// TC11 : mem_path + reg_write OFF
//-----

result_src_ex = 2'b01;
reg_write_in = 0;

#10;

```

```

if(result_src_mem == 2'b01 && reg_write_out == 1'b0)
    $display("ASSERT PASS : MEM_PATH_REGWRITE_OFF");
else
    $error("ASSERT FAIL : MEM_PATH_REGWRITE_OFF");

//-----
// TC12 : default_path + reg_write ON
//-----

result_src_ex = 2'b11;
reg_write_in = 1;

#10;

if(result_src_mem == 2'b11 && reg_write_out == 1'b1)
    $display("ASSERT PASS : DEFAULT_PATH_REGWRITE_ON");
else
    $error("ASSERT FAIL : DEFAULT_PATH_REGWRITE_ON");

//-----
// TC13 : default_path + reg_write OFF
//-----

result_src_ex = 2'b11;
reg_write_in = 0;

#10;

if(result_src_mem == 2'b11 && reg_write_out == 1'b0)
    $display("ASSERT PASS : DEFAULT_PATH_REGWRITE_OFF");
else
    $error("ASSERT FAIL : DEFAULT_PATH_REGWRITE_OFF");

//-----

```

```
// TC14 : PC4 Path + RegWrite OFF
```

```
//-----
```

```
$display("\nTC14 : PC4 Path + RegWrite OFF");
```

```
result_src_ex = 2'b10;
```

```
reg_write_in = 0;
```

```
#10;
```

```
if(result_src_mem == 2'b10 && reg_write_out == 1'b0)
```

```
    $display("ASSERT PASS : PC4_PATH_REGWRITE_OFF");
```

```
else
```

```
    $error("ASSERT FAIL : PC4_PATH_REGWRITE_OFF");
```

```
$display("\nEX_MEM VERIFICATION COMPLETE");
```

```
#10;
```

```
$display("\nCovrage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
$finish;
```

```
end
```

```
final begin
```

```
    $display("\nFinal Covrage = %0.2f%%",
```

```
        cg.get_coverage());
```

```
end
```

```
endmodule
```

9. Forwarding

```
`timescale 1ns/1ps
```

```
module tb_forwarding;
```

```
//=====
```

```
// INPUTS
```

```
//=====
```

```
reg [4:0] id_ex_rs1;
```

```
reg [4:0] id_ex_rs2;
```

```
reg [4:0] ex_mem_rd;
```

```
reg [4:0] mem_wb_rd;
```

```
reg ex_mem_reg_write;
```

```
reg mem_wb_reg_write;
```

```
//=====
```

```
// OUTPUTS
```

```
//=====
```

```
wire [1:0] forward_A;
```

```
wire [1:0] forward_B;
```

```
//=====
```

```
// DUT
```

```
//=====
```

```
forwarding dut(
```

```
    .id_ex_rs1(id_ex_rs1),
```

```
    .id_ex_rs2(id_ex_rs2),
```

```
    .ex_mem_rd(ex_mem_rd),
```

```

.mem_wb_rd(mem_wb_rd),

.ex_mem_reg_write(ex_mem_reg_write),
.mem_wb_reg_write(mem_wb_reg_write),

.forward_A(forward_A),
.forward_B(forward_B)
);

//=====
// COVERAGE
//=====

covergroup forwarding_cg @(forward_A or forward_B);

option.per_instance = 1;
option.name = "FORWARDING_COVERAGE";

cp_forward_A : coverpoint forward_A
{
    bins no_forward = {2'b00};
    bins ex_forward = {2'b01};
    bins mem_forward = {2'b10};
}

cp_forward_B : coverpoint forward_B
{
    bins no_forward = {2'b00};
    bins ex_forward = {2'b01};
    bins mem_forward = {2'b10};
}

endgroup

```



```
forwarding_cg cg = new();
```

```
//=====
```

```
// TESTCASES
```

```
//=====
```

```
initial begin
```

```
$dumpfile("dump.vcd");
```

```
$dumpvars(0,tb_forwarding);
```

```
//-----
```

```
// TC1 : NO FORWARDING
```

```
//-----
```

```
$display("\nTC1 : NO FORWARDING");
```

```
id_ex_rs1 = 5;
```

```
id_ex_rs2 = 6;
```

```
ex_mem_rd = 10;
```

```
mem_wb_rd = 11;
```

```
ex_mem_reg_write = 1;
```

```
mem_wb_reg_write = 1;
```

```
#5;
```

```
if(forward_A==2'b00 && forward_B==2'b00)
```

```
begin
```

```
    $display("PASS : TC1");
```

```
    $display("ASSERT PASS : NO FORWARDING");
```

```
end
```

```
else
```

```

    $error("ASSERT FAIL : TC1");

//-----
// TC2 : EX FORWARD A
//-----

$display("\nTC2 : EX FORWARD A");

id_ex_rs1 = 7;
ex_mem_rd = 7;

#5;

if(forward_A==2'b01)
begin
    $display("PASS : TC2");
    $display("ASSERT PASS : EX FORWARD A");
end
else
    $error("ASSERT FAIL : TC2");

//-----
// TC3 : EX FORWARD B
//-----

$display("\nTC3 : EX FORWARD B");

id_ex_rs2 = 12;
ex_mem_rd = 12;

#5;

if(forward_B==2'b01)
begin

```

```

    $display("PASS : TC3");
    $display("ASSERT PASS : EX FORWARD B");
end
else
    $error("ASSERT FAIL : TC3");

//-----
// TC4 : MEM FORWARD A
//-----

$display("\nTC4 : MEM FORWARD A");

id_ex_rs1 = 15;

ex_mem_reg_write = 0;

mem_wb_rd = 15;
mem_wb_reg_write = 1;

#5;

if(forward_A==2'b10)
begin
    $display("PASS : TC4");
    $display("ASSERT PASS : MEM FORWARD A");
end
else
    $error("ASSERT FAIL : TC4");

//-----
// TC5 : MEM FORWARD B
//-----

$display("\nTC5 : MEM FORWARD B");

```

```
id_ex_rs2 = 20;
```

```
mem_wb_rd = 20;
```

```
#5;
```

```
if(forward_B==2'b10)
```

```
begin
```

```
    $display("PASS : TC5");
```

```
    $display("ASSERT PASS : MEM FORWARD B");
```

```
end
```

```
else
```

```
    $error("ASSERT FAIL : TC5");
```

```
//-----
```

```
// TC6 : PRIORITY CHECK A
```

```
//-----
```

```
$display("\nTC6 : PRIORITY CHECK A");
```

```
id_ex_rs1 = 25;
```

```
ex_mem_rd = 25;
```

```
mem_wb_rd = 25;
```

```
ex_mem_reg_write = 1;
```

```
mem_wb_reg_write = 1;
```

```
#5;
```

```
if(forward_A==2'b01)
```

```
begin
```

```
    $display("PASS : TC6");
```

```

        $display("ASSERT PASS : EX PRIORITY");
    end
else
    $error("ASSERT FAIL : TC6");

//-----
// TC7 : PRIORITY CHECK B
//-----

$display("\nTC7 : PRIORITY CHECK B");

id_ex_rs2 = 30;

ex_mem_rd = 30;
mem_wb_rd = 30;

#5;

if(forward_B==2'b01)
begin
    $display("PASS : TC7");
    $display("ASSERT PASS : EX PRIORITY");
end
else
    $error("ASSERT FAIL : TC7");

//-----
// TC8 : ZERO REGISTER CHECK
//-----

$display("\nTC8 : ZERO REGISTER");

id_ex_rs1 = 0;

```

```

ex_mem_rd = 0;
mem_wb_rd = 0;

#5;

if(forward_A==2'b00)
begin
    $display("PASS : TC8");
    $display("ASSERT PASS : ZERO REGISTER");
end
else
    $error("ASSERT FAIL : TC8");

//-----

$display("\nFORWARDING VERIFICATION COMPLETE");

$display("\nCoverage = %0.2f%%",
    cg.get_coverage());

#10;
$finish;

end

final begin

$display("\nFinal Coverage = %0.2f%%",
    cg.get_coverage());

end

endmodule

```

10. Hazard Detection

```
`timescale 1ns/1ps
```

```
module tb_hazard_detection;
```

```
    // Inputs
```

```
    reg [4:0] rs1_id;
```

```
    reg [4:0] rs2_id;
```

```
    reg [4:0] rd_ex;
```

```
    reg MemRead_ex;
```

```
    // Outputs
```

```
    wire PC_Write;
```

```
    wire IF_ID_write;
```

```
    wire ID_EX_write;
```

```
    wire control_mux_sel;
```

```
    // DUT
```

```
    hazard_detection dut (
```

```
        .rs1_id(rs1_id),
```

```
        .rs2_id(rs2_id),
```

```
        .rd_ex(rd_ex),
```

```
        .MemRead_ex(MemRead_ex),
```

```
        .PC_Write(PC_Write),
```

```
        .IF_ID_write(IF_ID_write),
```

```
        .ID_EX_write(ID_EX_write),
```

```
        .control_mux_sel(control_mux_sel)
```

```
    );
```

```
    //-----
```

```
    // COVERAGE
```

```
    //-----
```

```
covergroup hazard_cg @(PC_Write or control_mux_sel);
```

```
    option.per_instance = 1;
```

```
    option.name = "HAZARD_COVERAGE";
```

```
    cp_pc_write : coverpoint PC_Write {
```

```
        bins stall = {0};
```

```
        bins normal = {1};
```

```
    }
```

```
    cp_control_mux : coverpoint control_mux_sel {
```

```
        bins nop_inserted = {1};
```

```
        bins normal_flow = {0};
```

```
    }
```

```
endgroup
```

```
hazard_cg cg = new();
```

```
//-----
```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
//-----
```

```
// TC1 : NO HAZARD
```

```
//-----
```

```
$display("\nTC1 : NO HAZARD");
```

```
    rs1_id  = 5'd1;
```

```
    rs2_id  = 5'd2;
```

```
    rd_ex   = 5'd3;
```

```
    MemRead_ex = 1'b1;
```



```
#5;
```

```
if(PC_Write==1 &&  
    IF_ID_write==1 &&  
    ID_EX_write==1 &&  
    control_mux_sel==0)  
begin  
    $display("PASS : TC1");  
    $display("ASSERT PASS : NO HAZARD");  
end  
else  
    $error("ASSERT FAIL : TC1");
```

```
//-----
```

```
// TC2 : RS1 HAZARD
```

```
//-----
```

```
$display("\nTC2 : RS1 HAZARD");
```

```
rs1_id   = 5'd5;
```

```
rs2_id   = 5'd2;
```

```
rd_ex    = 5'd5;
```

```
MemRead_ex = 1'b1;
```

```
#5;
```

```
if(PC_Write==0 &&  
    IF_ID_write==0 &&  
    ID_EX_write==0 &&  
    control_mux_sel==1)  
begin  
    $display("PASS : TC2");  
    $display("ASSERT PASS : RS1 HAZARD");  
end
```

```

else
    $error("ASSERT FAIL : TC2");

//-----
// TC3 : RS2 HAZARD
//-----
$display("\nTC3 : RS2 HAZARD");

rs1_id   = 5'd1;
rs2_id   = 5'd7;
rd_ex    = 5'd7;
MemRead_ex = 1'b1;

#5;

if(PC_Write==0 &&
    IF_ID_write==0 &&
    ID_EX_write==0 &&
    control_mux_sel==1)
begin
    $display("PASS : TC3");
    $display("ASSERT PASS : RS2 HAZARD");
end
else
    $error("ASSERT FAIL : TC3");

//-----
// TC4 : MEMREAD DISABLED
//-----
$display("\nTC4 : MEMREAD DISABLED");

rs1_id   = 5'd5;
rs2_id   = 5'd5;
rd_ex    = 5'd5;

```

```
MemRead_ex = 1'b0;
```

```
#5;
```

```
if(PC_Write==1 &&
```

```
    IF_ID_write==1 &&
```

```
    ID_EX_write==1 &&
```

```
    control_mux_sel==0)
```

```
begin
```

```
    $display("PASS : TC4");
```

```
    $display("ASSERT PASS : MEMREAD DISABLED");
```

```
end
```

```
else
```

```
    $error("ASSERT FAIL : TC4");
```

```
//-----
```

```
// TC5 : ZERO REGISTER
```

```
//-----
```

```
$display("\nTC5 : ZERO REGISTER");
```

```
rs1_id  = 5'd0;
```

```
rs2_id  = 5'd1;
```

```
rd_ex   = 5'd0;
```

```
MemRead_ex = 1'b1;
```

```
#5;
```

```
if(PC_Write==1 &&
```

```
    IF_ID_write==1 &&
```

```
    ID_EX_write==1 &&
```

```
    control_mux_sel==0)
```

```
begin
```

```
    $display("PASS : TC5");
```

```
    $display("ASSERT PASS : ZERO REGISTER");
```

```

end

else
    $error("ASSERT FAIL : TC5");

//-----
// TC6 : BOTH RS1 & RS2 MATCH
//-----
$display("\nTC6 : DOUBLE HAZARD");

rs1_id    = 5'd10;
rs2_id    = 5'd10;
rd_ex     = 5'd10;
MemRead_ex = 1'b1;

#5;

if(PC_Write==0 &&
    IF_ID_write==0 &&
    ID_EX_write==0 &&
    control_mux_sel==1)
begin
    $display("PASS : TC6");
    $display("ASSERT PASS : DOUBLE HAZARD");
end
else
    $error("ASSERT FAIL : TC6");

//-----
// TC7 : DYNAMIC SWITCHING
//-----
$display("\nTC7 : DYNAMIC SWITCHING");

// Normal
rs1_id    = 5'd1;

```

```

rs2_id    = 5'd2;
rd_ex     = 5'd8;
MemRead_ex = 1'b1;
#5;

assert(PC_Write==1)
    $display("ASSERT PASS : NORMAL");

// Hazard
rs1_id    = 5'd8;
rs2_id    = 5'd2;
rd_ex     = 5'd8;
MemRead_ex = 1'b1;
#5;

assert(PC_Write==0)
    $display("ASSERT PASS : HAZARD");

// Recovery
rs1_id    = 5'd1;
rs2_id    = 5'd2;
rd_ex     = 5'd20;
MemRead_ex = 1'b1;
#5;

assert(PC_Write==1)
    $display("ASSERT PASS : RECOVERY");
$display("\nHAZARD DETECTION VERIFICATION COMPLETE");
$display("\nCoverage = %0.2f%%", $get_coverage());
#10;
$finish;
end

endmodule

```

11. ID_EX

// Code your testbench here

// or browse Examples

//note: input id_ex_write signal is present in rtl but not used anywhere in the code so its testcase is not present in this

`timescale 1ns/1ps

module tb_ID_EX;

//=====

// INPUTS

//=====

reg clk;

reg rst;

reg stall;

reg flush;

reg id_ex_write;

reg [31:0] pc_id;

reg [4:0] rs1_id;

reg [4:0] rs2_id;

reg [4:0] rd_id;

reg [31:0] read_data1_id;

reg [31:0] read_data2_id;

reg [31:0] imm_val_id;

reg [2:0] funct3_id;

reg [6:0] funct7_id;

```

reg regWrite_id;
reg aluSrc_id;
reg [2:0] aluOp_id;
reg branch_id;
reg memWrite_id;
reg memRead_id;
reg memToReg_id;
reg jump_id;

reg [31:0] pc_plus_4_id;
reg [1:0] result_src_id;

//=====
// OUTPUTS
//=====

wire [31:0] pc_ex;

wire [4:0] rs1_ex;
wire [4:0] rs2_ex;
wire [4:0] rd_ex;

wire [31:0] read_data1_ex;
wire [31:0] read_data2_ex;
wire [31:0] imm_val_ex;

wire [2:0] funct3_ex;
wire [6:0] funct7_ex;

wire regWrite_ex;
wire aluSrc_ex;
wire [2:0] aluOp_ex;
wire branch_ex;
wire memWrite_ex;

```

```
wire memRead_ex;  
wire memToReg_ex;  
wire jump_ex;
```

```
wire [31:0] pc_plus_4_ex;  
wire [1:0] result_src_ex;
```

```
//=====
```

```
// DUT
```

```
//=====
```

```
ID_EX dut (
```

```
    .clk(clk),  
    .rst(rst),  
    .stall(stall),  
    .flush(flush),  
    .id_ex_write(id_ex_write),
```

```
    .pc_id(pc_id),
```

```
    .rs1_id(rs1_id),  
    .rs2_id(rs2_id),  
    .rd_id(rd_id),
```

```
    .read_data1_id(read_data1_id),  
    .read_data2_id(read_data2_id),  
    .imm_val_id(imm_val_id),
```

```
    .funct3_id(funct3_id),  
    .funct7_id(funct7_id),
```

```
    .regWrite_id(regWrite_id),  
    .aluSrc_id(aluSrc_id),
```


.aluOp_id(aluOp_id),
.branch_id(branch_id),
.memWrite_id(memWrite_id),
.memRead_id(memRead_id),
.memToReg_id(memToReg_id),
.jump_id(jump_id),

.pc_plus_4_id(pc_plus_4_id),
.result_src_id(result_src_id),

.pc_ex(pc_ex),

.rs1_ex(rs1_ex),
.rs2_ex(rs2_ex),
.rd_ex(rd_ex),

.read_data1_ex(read_data1_ex),
.read_data2_ex(read_data2_ex),
.imm_val_ex(imm_val_ex),

.funct3_ex(funct3_ex),
.funct7_ex(funct7_ex),

.regWrite_ex(regWrite_ex),
.aluSrc_ex(aluSrc_ex),
.aluOp_ex(aluOp_ex),
.branch_ex(branch_ex),
.memWrite_ex(memWrite_ex),
.memRead_ex(memRead_ex),
.memToReg_ex(memToReg_ex),
.jump_ex(jump_ex),

.pc_plus_4_ex(pc_plus_4_ex),
.result_src_ex(result_src_ex)

```
);
```

```
//-----  
// COVERAGE  
//-----
```

```
covergroup id_ex_cg @(posedge clk);
```

```
option.per_instance = 1;
```

```
option.name = "ID_EX_COVERAGE";
```

```
cp_rst : coverpoint rst
```

```
{  
    bins reset_on = {1};  
    bins reset_off = {0};  
}
```

```
cp_flush : coverpoint flush
```

```
{  
    bins flush_on = {1};  
    bins flush_off = {0};  
}
```

```
cp_stall : coverpoint stall
```

```
{  
    bins stall_on = {1};  
    bins stall_off = {0};  
}
```

```
cp_regwrite : coverpoint regWrite_ex
```

```
{  
    bins enabled = {1};  
    bins disabled = {0};  
}
```

```
cp_result_src : coverpoint result_src_ex
```

```
{
```

```
    bins alu_path = {2'b00};
```

```
    bins mem_path = {2'b01};
```

```
    bins pc4_path = {2'b10};
```

```
}
```

```
endgroup
```

```
id_ex_cg cg = new();
```

```
//=====
```

```
// CLOCK
```

```
//=====
```

```
always #5 clk = ~clk;
```

```
//ASSERTIONS
```

```
//-----
```

```
// RESET PROPERTY
```

```
//-----
```

```
property p_reset;
```

```
@(posedge clk)
```

```
rst |=> (
```

```
pc_ex == 0 &&
```

```
rs1_ex == 0 &&
```

```
rs2_ex == 0 &&
```

```
rd_ex == 0 &&
```

```
regWrite_ex == 0 &&
```

```
aluSrc_ex == 0 &&
```

```
aluOp_ex == 0 &&
```

```

branch_ex == 0 &&
memWrite_ex == 0 &&
memRead_ex == 0 &&
memToReg_ex == 0 &&
jump_ex == 0 &&
result_src_ex == 0
);
endproperty

```

```

assert property(p_reset)
$display("ASSERT PASS : RESET");
else
$error("ASSERT FAIL : RESET");

```

```

//-----
// FLUSH PROPERTY
//-----

```

```

property p_flush;
@(posedge clk)
(!rst && flush) | => (
pc_ex == 0 &&
rs1_ex == 0 &&
rs2_ex == 0 &&
rd_ex == 0 &&
regWrite_ex == 0 &&
aluSrc_ex == 0 &&
aluOp_ex == 0 &&
branch_ex == 0 &&
memWrite_ex == 0 &&
memRead_ex == 0 &&
memToReg_ex == 0 &&
jump_ex == 0 &&
result_src_ex == 0
)

```

```

);
endproperty

assert property(p_flush)
$display("ASSERT PASS : FLUSH");
else
$error("ASSERT FAIL : FLUSH");

//-----
// STALL PROPERTY
//-----

property p_stall;
@(posedge clk)
(!rst && stall) | => (
regWrite_ex == 0 &&
aluSrc_ex  == 0 &&
aluOp_ex   == 0 &&
branch_ex  == 0 &&
memWrite_ex == 0 &&
memRead_ex == 0 &&
memToReg_ex == 0 &&
jump_ex    == 0 &&
result_src_ex == 0
);
endproperty

assert property(p_stall)
$display("ASSERT PASS : STALL");
else
$error("ASSERT FAIL : STALL");

//-----
// NORMAL TRANSFER PROPERTY

```

```
//-----
```

```
property p_transfer;
@(posedge clk)
(!rst && !flush && !stall)
|=> (
pc_ex      == $past(pc_id) &&
rs1_ex     == $past(rs1_id) &&
rs2_ex     == $past(rs2_id) &&
rd_ex      == $past(rd_id) &&
read_data1_ex == $past(read_data1_id) &&
read_data2_ex == $past(read_data2_id) &&
imm_val_ex  == $past(imm_val_id) &&
funct3_ex  == $past(funct3_id) &&
funct7_ex  == $past(funct7_id) &&
regWrite_ex == $past(regWrite_id) &&
aluSrc_ex  == $past(aluSrc_id) &&
aluOp_ex   == $past(aluOp_id) &&
branch_ex  == $past(branch_id) &&
memWrite_ex == $past(memWrite_id) &&
memRead_ex == $past(memRead_id) &&
memToReg_ex == $past(memToReg_id) &&
jump_ex    == $past(jump_id) &&
pc_plus_4_ex == $past(pc_plus_4_id) &&
result_src_ex == $past(result_src_id)
);
endproperty
```

```
assert property(p_transfer)
$display("ASSERT PASS : TRANSFER");
else
$error("ASSERT FAIL : TRANSFER");
```

```
//=====
```

```
// TESTCASES
```

```
//=====
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_ID_EX);
```

```
    //-----
```

```
    // Initialize
```

```
    //-----
```

```
    clk = 0;
```

```
    rst = 0;
```

```
    stall = 0;
```

```
    flush = 0;
```

```
    id_ex_write = 1;
```

```
    pc_id = 0;
```

```
    rs1_id = 0;
```

```
    rs2_id = 0;
```

```
    rd_id = 0;
```

```
    read_data1_id = 0;
```

```
    read_data2_id = 0;
```

```
    imm_val_id = 0;
```

```
    funct3_id = 0;
```

```
    funct7_id = 0;
```

```
    regWrite_id = 0;
```

```
    aluSrc_id = 0;
```

```
    aluOp_id = 0;
```

```

branch_id = 0;
memWrite_id = 0;
memRead_id = 0;
memToReg_id = 0;
jump_id = 0;

pc_plus_4_id = 0;
result_src_id = 0;

// Paste TC1-TC8 here

//-----
// TC1 : RESET
//-----

$display("\nTC1 : RESET");

rst = 1;

#12;

if(
    pc_ex == 0 &&
    rs1_ex == 0 &&
    rs2_ex == 0 &&
    rd_ex == 0 &&
    regWrite_ex == 0 &&
    aluSrc_ex == 0 &&
    result_src_ex == 0
)
    $display("PASS : TC1");
else
    $display("FAIL : TC1");

```



```

//-----
// TC2 : NORMAL TRANSFER
//-----

$display("\nTC2 : NORMAL TRANSFER");

rst = 0;

pc_id = 100;

rs1_id = 5;
rs2_id = 10;
rd_id = 15;

read_data1_id = 111;
read_data2_id = 222;

imm_val_id = 333;

funct3_id = 3'b101;
funct7_id = 7'b0100000;

regWrite_id = 1;
aluSrc_id = 1;
aluOp_id = 3'b010;

branch_id = 1;
memWrite_id = 1;
memRead_id = 1;
memToReg_id = 1;
jump_id = 1;

pc_plus_4_id = 104;

```

```
result_src_id = 2'b01;
```

```
#10;
```

```
if(
```

```
    pc_ex == 100 &&
```

```
    rs1_ex == 5 &&
```

```
    rs2_ex == 10 &&
```

```
    rd_ex == 15 &&
```

```
    read_data1_ex == 111 &&
```

```
    read_data2_ex == 222 &&
```

```
    imm_val_ex == 333 &&
```

```
    pc_plus_4_ex == 104 &&
```

```
    result_src_ex == 2'b01
```

```
)
```

```
    $display("PASS : TC2");
```

```
else
```

```
    $display("FAIL : TC2");
```

```
//-----
```

```
// TC3 : STALL
```

```
//-----
```

```
$display("\nTC3 : STALL");
```

```
stall = 1;
```

```
pc_id = 999;
```

```
rd_id = 31;
```

```
result_src_id = 2'b10;
```

```
#10;
```

```
if(
```

```

    pc_ex == 100 &&
    rd_ex == 15 &&
    regWrite_ex == 0 &&
    memWrite_ex == 0 &&
    memRead_ex == 0 &&
    result_src_ex == 0
)
    $display("PASS : TC3");
else
    $display("FAIL : TC3");

stall = 0;

//-----
// TC4 : FLUSH
//-----

$display("\nTC4 : FLUSH");

flush = 1;

#10;

if(
    pc_ex == 0 &&
    rs1_ex == 0 &&
    rs2_ex == 0 &&
    rd_ex == 0 &&
    regWrite_ex == 0 &&
    result_src_ex == 0
)
    $display("PASS : TC4");
else
    $display("FAIL : TC4");

```

```

flush = 0;

//-----
// TC5 : FLUSH PRIORITY OVER STALL
//-----

$display("\nTC5 : FLUSH PRIORITY");

flush = 1;
stall = 1;

#10;

if(
    pc_ex == 0 &&
    rd_ex == 0 &&
    regWrite_ex == 0
)
    $display("PASS : TC5");
else
    $display("FAIL : TC5");

flush = 0;
stall = 0;

//-----
// TC6 : result_src = MEM
//-----

$display("\nTC6 : MEM PATH");

result_src_id = 2'b01;

```

```

#10;

if(result_src_ex == 2'b01)
    $display("PASS : TC6");
else
    $display("FAIL : TC6");

//-----
// TC7 : result_src = PC+4
//-----

$display("\nTC7 : PC+4 PATH");

result_src_id = 2'b10;

#10;

if(result_src_ex == 2'b10)
    $display("PASS : TC7");
else
    $display("FAIL : TC7");

//-----
// TC8 : SECOND NORMAL TRANSFER
//-----

$display("\nTC8 : SECOND TRANSFER");
pc_id = 500;
rs1_id = 20;
rs2_id = 21;
rd_id = 22;
read_data1_id = 1000;
read_data2_id = 2000;
imm_val_id = 3000;

```

```

pc_plus_4_id = 504;
result_src_id = 2'b00;
regWrite_id = 1;
aluSrc_id = 1;

#10;

if(
    pc_ex == 500 &&
    rd_ex == 22 &&
    read_data1_ex == 1000 &&
    read_data2_ex == 2000 &&
    imm_val_ex == 3000 &&
    pc_plus_4_ex == 504
)
    $display("PASS : TC8");
else
    $display("FAIL : TC8");

//-----

$display("\nID_EX VERIFICATION COMPLETE");

#10;
$finish;
end

final begin

$display("\nFinal Coverage = %0.2f%%",
    cg.get_coverage());

end

endmodule

```

12.IF_ID

```
// Code your testbench here
```

```
// or browse Examples
```

```
`timescale 1ns/1ps
```

```
module tb_IF_ID;
```

```
reg clk;
```

```
reg rst;
```

```
reg IF_flush;
```

```
reg IF_ID_write;
```

```
reg [31:0] IF_instr;
```

```
reg [31:0] IF_pc;
```

```
reg [31:0] IF_pcplus4;
```

```
wire [31:0] Instr_ID;
```

```
wire [31:0] pc_ID;
```

```
wire [31:0] pc_plus_4_ID;
```

```
IF_ID dut(
```

```
    .clk(clk),
```

```
    .rst(rst),
```

```
    .IF_flush(IF_flush),
```

```
    .IF_ID_write(IF_ID_write),
```

```
    .IF_instr(IF_instr),
```

```
    .IF_pc(IF_pc),
```

```
    .IF_pcplus4(IF_pcplus4),
```

```
    .Instr_ID(Instr_ID),
```

```

        .pc_ID(pc_ID),
        .pc_plus_4_ID(pc_plus_4_ID)
    );

//-----
// CLOCK
//-----

always #5 clk = ~clk;

//-----
// COVERAGE
//-----

covergroup ifid_cg @(posedge clk);

    option.per_instance = 1;
    option.name = "IF_ID_COVERAGE";

    cp_rst : coverpoint rst
    {
        bins reset_on = {1};
        bins reset_off = {0};
    }

    cp_flush : coverpoint IF_flush
    {
        bins flush_on = {1};
        bins flush_off = {0};
    }

    cp_write : coverpoint IF_ID_write
    {
        bins write_on = {1};

```



```

        bins write_off = {0};
    }

endgroup

ifid_cg cg = new();

//Assertions

property p_reset;

@(posedge clk)

rst |=> (
    Instr_ID == 0 &&
    pc_ID == 0 &&
    pc_plus_4_ID == 0
);

endproperty

assert property(p_reset);

property p_transfer;

@(posedge clk)

(!rst && !IF_flush && IF_ID_write)

|=>

Instr_ID == $past(IF_instr) &&
pc_ID == $past(IF_pc) &&
pc_plus_4_ID == $past(IF_pcplus4);

```

```
endproperty
```

```
assert property(p_transfer);
```

```
property p_stall;
```

```
@(posedge clk)
```

```
(!rst && !IF_flush && !IF_ID_write)
```

```
==>
```

```
Instr_ID == $past(Instr_ID) &&
```

```
pc_ID == $past(pc_ID) &&
```

```
pc_plus_4_ID == $past(pc_plus_4_ID);
```

```
endproperty
```

```
assert property(p_stall);
```

```
property p_flush;
```

```
@(posedge clk)
```

```
IF_flush
```

```
==>
```

```
Instr_ID == 0 &&
```

```
pc_ID == 0 &&
```

```
pc_plus_4_ID == 0;
```

```
endproperty
```

```
assert property(p_flush);
```

```
//-----
```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_IF_ID);
```

```
    clk = 0;
```

```
//-----
```

```
// TC1 : RESET
```

```
//-----
```

```
    $display("\nTC1 : RESET");
```

```
    rst = 1;
```

```
    IF_flush = 0;
```

```
    IF_ID_write = 0;
```

```
    IF_instr = 32'h12345678;
```

```
    IF_pc = 32'h100;
```

```
    IF_pcplus4 = 32'h104;
```

```
    #12;
```

```
    if(
```

```
        Instr_ID == 0 &&
```

```
        pc_ID == 0 &&
```

```
        pc_plus_4_ID == 0
```

```

)
    $display("PASS : TC1");
else
    $display("FAIL : TC1");

if(
    Instr_ID == 0 &&
    pc_ID == 0 &&
    pc_plus_4_ID == 0
)
    $display("ASSERT PASS : RESET");
else
    $error("ASSERT FAIL : RESET");

//-----
// TC2 : NORMAL TRANSFER
//-----

$display("\nTC2 : NORMAL TRANSFER");

rst = 0;
IF_flush = 0;
IF_ID_write = 1;

IF_instr = 32'hAABBCCDD;
IF_pc = 32'h200;
IF_pcplus4 = 32'h204;

#10;

if(
    Instr_ID == 32'hAABBCCDD &&
    pc_ID == 32'h200 &&
    pc_plus_4_ID == 32'h204

```

```

)
    $display("PASS : TC2");
else
    $display("FAIL : TC2");

if(
    Instr_ID == 32'hAABBCCDD &&
    pc_ID == 32'h200 &&
    pc_plus_4_ID == 32'h204
)
    $display("ASSERT PASS : TRANSFER");
else
    $error("ASSERT FAIL : TRANSFER");

//-----
// TC3 : STALL
//-----

$display("\nTC3 : STALL");

IF_ID_write = 0;

IF_instr = 32'hFFFFFFF;
IF_pc = 32'h999;
IF_pcplus4 = 32'h999;

#10;

if(
    Instr_ID == 32'hAABBCCDD &&
    pc_ID == 32'h200 &&
    pc_plus_4_ID == 32'h204
)
    $display("PASS : TC3");

```

```

else
    $display("FAIL : TC3");

if(
    Instr_ID == 32'hAABBCCDD &&
    pc_ID == 32'h200 &&
    pc_plus_4_ID == 32'h204
)
    $display("ASSERT PASS : STALL");
else
    $error("ASSERT FAIL : STALL");

//-----
// TC4 : FLUSH
//-----

$display("\nTC4 : FLUSH");

IF_flush = 1;

#10;

if(
    Instr_ID == 0 &&
    pc_ID == 0 &&
    pc_plus_4_ID == 0
)
    $display("PASS : TC4");
else
    $display("FAIL : TC4");

if(
    Instr_ID == 0 &&
    pc_ID == 0 &&

```

```

    pc_plus_4_ID == 0
)
    $display("ASSERT PASS : FLUSH");
else
    $error("ASSERT FAIL : FLUSH");

//-----
// TC5 : FLUSH PRIORITY OVER WRITE
//-----

$display("\nTC5 : FLUSH PRIORITY");

IF_flush = 1;
IF_ID_write = 1;

IF_instr = 32'hDEADBEEF;
IF_pc = 32'h888;
IF_pcplus4 = 32'h88C;

#10;

if(
    Instr_ID == 0 &&
    pc_ID == 0 &&
    pc_plus_4_ID == 0
)
    $display("PASS : TC5");
else
    $display("FAIL : TC5");

if(
    Instr_ID == 0 &&
    pc_ID == 0 &&
    pc_plus_4_ID == 0

```

```

)
    $display("ASSERT PASS : FLUSH PRIORITY");
else
    $error("ASSERT FAIL : FLUSH PRIORITY");

//-----
// TC6 : NORMAL AGAIN
//-----

$display("\nTC6 : NORMAL AGAIN");

IF_flush = 0;
IF_ID_write = 1;

IF_instr = 32'hCAFEFABE;
IF_pc = 32'h300;
IF_pcplus4 = 32'h304;

#10;

if(
    Instr_ID == 32'hCAFEFABE &&
    pc_ID == 32'h300 &&
    pc_plus_4_ID == 32'h304
)
    $display("PASS : TC6");
else
    $display("FAIL : TC6");

if(
    Instr_ID == 32'hCAFEFABE &&
    pc_ID == 32'h300 &&
    pc_plus_4_ID == 32'h304
)

```



```
        $display("ASSERT PASS : NORMAL AGAIN");
    else
        $error("ASSERT FAIL : NORMAL AGAIN");

    //-----

    $display("\nIF_ID VERIFICATION COMPLETE");

    $display("\nCoverage = %0.2f%%",
        cg.get_coverage());

    #10;
    $finish;

end

final begin

    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());

end

endmodule
```

13. imm_gen

```
`timescale 1ns/1ps

module tb_imm_gen;

    reg [31:0] instruction;
    wire [31:0] imm_val;

    //=====
    // DUT
    //=====

    imm_gen dut (
        .instruction(instruction),
        .imm_val(imm_val)
    );

    //=====
    // COVERAGE
    //=====

    covergroup imm_gen_cg @(instruction);

        option.per_instance = 1;
        option.name = "IMM_GEN_COVERAGE";

        cp_opcode : coverpoint instruction[6:0]
        {
            bins i_type = {7'b0010011};
            bins load   = {7'b0000011};
            bins jalr    = {7'b1100111};

            bins store   = {7'b0100011};
```

```

    bins branch = {7'b1100011};

    bins lui    = {7'b0110111};
    bins auipc  = {7'b0010111};

    bins jal    = {7'b1101111};

    bins invalid = default;
}

cp_imm : coverpoint imm_val
{
    bins zero_val  = {0};

    bins small_pos = {[1:100]};

    bins medium_pos = {[101:1000]};

    bins large_pos  = {[1001:$]};
}

endgroup

imm_gen_cg cg = new();

//=====
// TESTCASES
//=====

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_imm_gen);

```

```

//-----
// TC1 : I-TYPE
//-----

$display("\nTC1 : I-TYPE");

instruction = 32'h06400093;

#5;
if(imm_val == 100)
    $display("ASSERT PASS : I_TYPE");
else
    $error("ASSERT FAIL : I_TYPE");

$display("IMM = %0d",imm_val);

//-----
// TC2 : LOAD
//-----

$display("\nTC2 : LOAD");

instruction = 32'h02002103;

#5;

if(imm_val == 32)
    $display("ASSERT PASS : LOAD");
else
    $error("ASSERT FAIL : LOAD");

$display("IMM = %0d",imm_val);

```

```

//-----
// TC3 : JALR
//-----

$display("\nTC3 : JALR");

instruction = 32'h01008067;

#5;

if(imm_val == 16)
    $display("ASSERT PASS : JALR");
else
    $error("ASSERT FAIL : JALR");

$display("IMM = %0d",imm_val);

//-----
// TC4 : STORE
//-----

$display("\nTC4 : STORE");

instruction = 32'h06202223;

#5;

if(imm_val == 100)
    $display("ASSERT PASS : STORE");
else
    $error("ASSERT FAIL : STORE");

$display("IMM = %0d",imm_val);

```

```

//-----
// TC5 : BRANCH
//-----

$display("\nTC5 : BRANCH");

instruction = 32'h00208863;

#5;

if(imm_val == 16)
    $display("ASSERT PASS : BRANCH");
else
    $error("ASSERT FAIL : BRANCH");

$display("IMM = %0d",imm_val);

//-----
// TC6 : LUI
//-----

$display("\nTC6 : LUI");

instruction = 32'h12345037;

#5;

if(imm_val == 32'h12345000)
    $display("ASSERT PASS : LUI");
else
    $error("ASSERT FAIL : LUI");

$display("IMM = %0d",imm_val);

```

```

//-----
// TC7 : AUIPC
//-----

$display("\nTC7 : AUIPC");

instruction = 32'h12345017;

#5;

if(imm_val == 32'h12345000)
    $display("ASSERT PASS : AUIPC");
else
    $error("ASSERT FAIL : AUIPC");

$display("IMM = %0d",imm_val);

//-----
// TC8 : JAL
//-----

$display("\nTC8 : JAL");

instruction = 32'h020000EF;

#5;

if(imm_val == 32)
    $display("ASSERT PASS : JAL");
else
    $error("ASSERT FAIL : JAL");

$display("IMM = %0d",imm_val);

```

```

//-----
// TC9 : INVALID OPCODE
//-----

$display("\nTC9 : INVALID");

instruction = 32'hFFFFFFFF;

#5;

if(imm_val == 0)
    $display("ASSERT PASS : INVALID OPCODE");
else
    $error("ASSERT FAIL : INVALID OPCODE");

$display("IMM = %0d",imm_val);

//-----
// TC10 : ZERO IMM
//-----

$display("\nTC10 : ZERO IMM");

instruction = 32'h00000013; // addi x0,x0,0

#5;

if(imm_val == 0)
    $display("ASSERT PASS : ZERO IMM");
else
    $error("ASSERT FAIL : ZERO IMM");

$display("IMM = %0d", imm_val);

```



```

//-----
// TC11 : MEDIUM IMM
//-----

$display("\nTC11 : MEDIUM IMM");

instruction = 32'h1F400093; // imm = 500

#5;

if(imm_val == 500)
    $display("ASSERT PASS : MEDIUM IMM");
else
    $error("ASSERT FAIL : MEDIUM IMM");

$display("IMM = %0d", imm_val);

//-----
// TC12 : FORCE MEDIUM BIN
//-----

$display("\nTC12 : FORCE MEDIUM BIN");

instruction = 32'h32000093; // imm = 800

#10;

if(imm_val == 800)
    $display("ASSERT PASS : MEDIUM IMM");
else
    $error("ASSERT FAIL : MEDIUM IMM");

$display("IMM = %0d", imm_val);

```

```
//-----
```

```
$display("\nIMM_GEN VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",  
        cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
$display("\nFinal Coverage = %0.2f%%",  
        cg.get_coverage());
```

```
end
```

```
endmodule
```

14.Instruction Memory

```
`timescale 1ns/1ps

module tb_instruction_memory;

    reg [31:0] pc;
    wire [31:0] instruction_code;

    instruction_memory dut(
        .pc(pc),
        .instruction_code(instruction_code)
    );

    //-----
    // COVERAGE
    //-----

    covergroup instr_mem_cg @(pc);

        option.per_instance = 1;
        option.name = "INSTR_MEM_COVERAGE";

        cp_pc : coverpoint pc[31:2]
        {
            bins addr0 = {0};
            bins addr1 = {1};
            bins addr2 = {2};
            bins addr3 = {3};
        }

    endgroup

    instr_mem_cg cg = new();
```

```
//-----  
// TESTCASES  
//-----  
  
initial begin  
  
    $dumpfile("dump.vcd");  
    $dumpvars(0,tb_instruction_memory);  
  
    //-----  
    // TC1 : Address 0  
    //-----  
  
    $display("\nTC1 : Address 0");  
  
    pc = 32'd0;  
  
    #5;  
  
    if(instruction_code == 32'h00000013)  
        $display("PASS : TC1");  
    else  
        $display("FAIL : TC1");  
  
    if(instruction_code == 32'h00000013)  
        $display("ASSERT PASS : ADDR0");  
    else  
        $error("ASSERT FAIL : ADDR0");  
  
    //-----  
    // TC2 : Address 1  
    //-----
```

```
$display("\nTC2 : Address 1");
```

```
pc = 32'd4;
```

```
#5;
```

```
if(instruction_code == 32'h00100093)
```

```
    $display("PASS : TC2");
```

```
else
```

```
    $display("FAIL : TC2");
```

```
if(instruction_code == 32'h00100093)
```

```
    $display("ASSERT PASS : ADDR1");
```

```
else
```

```
    $error("ASSERT FAIL : ADDR1");
```

```
//-----
```

```
// TC3 : Address 2
```

```
//-----
```

```
$display("\nTC3 : Address 2");
```

```
pc = 32'd8;
```

```
#5;
```

```
if(instruction_code == 32'h00200113)
```

```
    $display("PASS : TC3");
```

```
else
```

```
    $display("FAIL : TC3");
```

```
if(instruction_code == 32'h00200113)
```

```
    $display("ASSERT PASS : ADDR2");
```

```
else
```

```

    $error("ASSERT FAIL : ADDR2");

//-----
// TC4 : Address 3
//-----

$display("\nTC4 : Address 3");

pc = 32'd12;

#5;

if(instruction_code == 32'h00308193)
    $display("PASS : TC4");
else
    $display("FAIL : TC4");

if(instruction_code == 32'h00308193)
    $display("ASSERT PASS : ADDR3");
else
    $error("ASSERT FAIL : ADDR3");

//-----
// TC5 : Alignment Check
//-----

$display("\nTC5 : Alignment Check");

pc = 32'd1;

#5;

if(instruction_code == 32'h00000013)
    $display("PASS : TC5");

```

```

else
    $display("FAIL : TC5");

//-----
// TC6 : Another Alignment Check
//-----

$display("\nTC6 : Alignment Check");

pc = 32'd2;

#5;

if(instruction_code == 32'h00000013)
    $display("PASS : TC6");
else
    $display("FAIL : TC6");

//-----
// TC7 : Another Alignment Check
//-----

$display("\nTC7 : Alignment Check");

pc = 32'd3;

#5;

if(instruction_code == 32'h00000013)
    $display("PASS : TC7");
else
    $display("FAIL : TC7");

//-----

```

```
$display("\nINSTRUCTION_MEMORY VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",  
        cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
$display("\nFinal Coverage = %0.2f%%",  
        cg.get_coverage());
```

```
end
```

```
endmodule
```


15.Mem_wb

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_mem_wb;

    reg clk;
    reg rst;

    reg [31:0] alu_result_mem;
    reg [31:0] read_data_mem;
    reg reg_write_mem;
    reg mem_to_reg_mem;
    reg [4:0] rd_mem;

    reg [31:0] pc_plus_4_mem;
    reg [1:0] result_src_mem;

    wire [31:0] alu_result_wb;
    wire [31:0] read_data_wb;
    wire reg_write_wb;
    wire mem_to_reg_wb;
    wire [4:0] rd_wb;

    wire [31:0] pc_plus_4_wb;
    wire [1:0] result_src_wb;

    mem_wb dut(
        .clk(clk),
        .rst(rst),

        .alu_result_mem(alu_result_mem),
```

```

.read_data_mem(read_data_mem),
.reg_write_mem(reg_write_mem),
.mem_to_reg_mem(mem_to_reg_mem),
.rd_mem(rd_mem),

.alu_result_wb(alu_result_wb),
.read_data_wb(read_data_wb),
.reg_write_wb(reg_write_wb),
.mem_to_reg_wb(mem_to_reg_wb),
.rd_wb(rd_wb),

.pc_plus_4_mem(pc_plus_4_mem),
.pc_plus_4_wb(pc_plus_4_wb),

.result_src_mem(result_src_mem),
.result_src_wb(result_src_wb)
);

//-----
// Functional Coverage
//-----

covergroup mem_wb_cg @(posedge clk);

option.per_instance = 1;
option.name = "MEM_WB_COVERAGE";

// Reset coverage
cp_rst : coverpoint rst {
    bins asserted  = {1};
    bins deasserted = {0};
}

// Control coverage

```

```

cp_reg_write : coverpoint reg_write_mem {
    bins off = {0};
    bins on  = {1};
}

```

```

cp_mem_to_reg : coverpoint mem_to_reg_mem {
    bins off = {0};
    bins on  = {1};
}

```

// Result source coverage

```

cp_result_src : coverpoint result_src_mem {
    bins alu_path = {2'b00};
    bins mem_path = {2'b01};
    bins pc4_path = {2'b10};
    bins default_path = {2'b11};
}

```

// Destination register coverage

```

cp_rd : coverpoint rd_mem {

    bins x0    = {0};
    bins low_reg = {[1:10]};
    bins mid_reg = {[11:20]};
    bins high_reg = {[21:31]};
}

```

// Cross coverage

```

cross_result_regwrite : cross cp_result_src, cp_reg_write;

```

endgroup

```

mem_wb_cg cg = new();

```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_mem_wb);
```

```
    clk = 0;
```

```
    //-----
```

```
    // TC1 RESET
```

```
    //-----
```

```
    rst = 1;
```

```
    alu_result_mem = 32'd100;
```

```
    read_data_mem = 32'd200;
```

```
    rd_mem = 5;
```

```
    reg_write_mem = 1;
```

```
    mem_to_reg_mem = 1;
```

```
    pc_plus_4_mem = 32'h24;
```

```
    result_src_mem = 2'b10;
```

```
    #12;
```

```
    if(
```

```
        alu_result_wb==0 &&
```

```
        read_data_wb==0 &&
```

```
        rd_wb==0 &&
```

```

        reg_write_wb==0 &&
        mem_to_reg_wb==0 &&
        pc_plus_4_wb==0 &&
        result_src_wb==0
    )
    $display("PASS : TC1 Reset");
else
    $display("FAIL : TC1 Reset");

//-----
// TC2 DATA PATH
//-----

rst = 0;

alu_result_mem = 32'd111;
read_data_mem = 32'd222;
rd_mem = 5;

#10;

if(alu_result_wb == alu_result_mem)
    $display("ASSERT PASS : ALU TRANSFER");
else
    $error("ASSERT FAIL : ALU TRANSFER");

if(
    alu_result_wb==111 &&
    read_data_wb==222 &&
    rd_wb==5
)
    $display("PASS : TC2 Data Path");
else
    $display("FAIL : TC2 Data Path");

```

```

//-----
// TC3 CONTROL PATH
//-----

reg_write_mem = 1;
mem_to_reg_mem = 1;

#10;

if(reg_write_wb == reg_write_mem)
    $display("ASSERT PASS : REG_WRITE");
else
    $error("ASSERT FAIL : REG_WRITE");

if(mem_to_reg_wb == mem_to_reg_mem)
    $display("ASSERT PASS : MEM_TO_REG");
else
    $error("ASSERT FAIL : MEM_TO_REG");

if(
    reg_write_wb==1 &&
    mem_to_reg_wb==1
)
    $display("PASS : TC3 Control Path");
else
    $display("FAIL : TC3 Control Path");

//-----
// TC4 PC+4
//-----

pc_plus_4_mem = 32'h00000024;

```

```

#10;

if(pc_plus_4_wb == pc_plus_4_mem)
    $display("ASSERT PASS : PC4");
else
    $error("ASSERT FAIL : PC4");

if(pc_plus_4_wb==32'h00000024)
    $display("PASS : TC4 PC+4");
else
    $display("FAIL : TC4 PC+4");

//-----
// TC5 RESULT_SRC
//-----

result_src_mem = 2'b10;

#10;

if(result_src_wb == result_src_mem)
    $display("ASSERT PASS : RESULT_SRC");
else
    $error("ASSERT FAIL : RESULT_SRC");

if(result_src_wb==2'b10)
    $display("PASS : TC5 result_src");
else
    $display("FAIL : TC5 result_src");

//-----
// TC6 UPDATE TEST
//-----

```

```

alu_result_mem = 32'd555;
read_data_mem = 32'd777;
rd_mem = 10;

#10;

if(rd_wb == rd_mem)
    $display("ASSERT PASS : RD");
else
    $error("ASSERT FAIL : RD");

if(
    alu_result_wb==555 &&
    read_data_wb==777 &&
    rd_wb==10
)
    $display("PASS : TC6 Update Test");
else
    $display("FAIL : TC6 Update Test");

//-----
// TC7 : ALU PATH + OFF CONTROL
//-----

reg_write_mem = 0;
mem_to_reg_mem = 0;

result_src_mem = 2'b00;
rd_mem = 0;

#10;

//-----
// TC8 : MEM PATH

```



```
//-----
```

```
result_src_mem = 2'b01;
```

```
rd_mem      = 15;
```

```
#10;
```

```
//-----
```

```
// TC9 : DEFAULT PATH
```

```
//-----
```

```
result_src_mem = 2'b11;
```

```
rd_mem      = 25;
```

```
#10;
```

```
//-----
```

```
// TC10 : HIGH REG + CONTROL ON
```

```
//-----
```

```
reg_write_mem = 1;
```

```
mem_to_reg_mem = 1;
```

```
result_src_mem = 2'b00;
```

```
rd_mem      = 31;
```

```
#10;
```

```
//-----
```

```
// TC11 : Cross Coverage
```

```
//-----
```

```
result_src_mem = 2'b01;
```

```
reg_write_mem = 1;
```

```
#10;
```

```
//-----  
// TC12 : Cross Coverage  
//-----
```

```
result_src_mem = 2'b10;
```

```
reg_write_mem = 0;
```

```
#10;
```

```
//-----  
// TC13 : Cross Coverage  
//-----
```

```
result_src_mem = 2'b11;
```

```
reg_write_mem = 1;
```

```
#10;
```

```
$display("\nMEM_WB VERIFICATION COMPLETE");
```

```
#10;
```

```
$display("\nCovrage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
$finish;
```

```
end
```

```
final begin
```

```
$display("\nFinal Covrage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
end
```

```
endmodule
```

16.Mux_wb

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_mux_wb;

    reg [31:0] alu_result;
    reg [31:0] r_data;
    reg [31:0] pc_plus_4_wb;
    reg [1:0] result_src;

    wire [31:0] data_out;

    mux_wb dut(
        .alu_result(alu_result),
        .r_data(r_data),
        .pc_plus_4_wb(pc_plus_4_wb),
        .result_src(result_src),
        .data_out(data_out)
    );

    //-----
    // Functional Coverage
    //-----

    covergroup mux_wb_cg @(result_src);

        option.per_instance = 1;
        option.name = "MUX_WB_COVERAGE";

        cp_result_src : coverpoint result_src {
```

```

        bins alu_path    = {2'b00};
        bins mem_path    = {2'b01};
        bins pc4_path     = {2'b10};
        bins default_path = {2'b11};

    }

endgroup

mux_wb_cg cg = new();

//assertions

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_mux_wb);

    //-----
    // Common Inputs
    //-----

    alu_result  = 32'd100;
    r_data      = 32'd200;
    pc_plus_4_wb = 32'd300;

    //-----
    // TC1 : ALU RESULT SELECT
    //-----

    $display("\nTC1 : ALU Result Select");

    result_src = 2'b00;

```

```
#5;
```

```
assert(data_out == alu_result)
```

```
    $display("ASSERT PASS : ALU PATH");
```

```
else
```

```
    $error("ASSERT FAIL : ALU PATH");
```

```
if(data_out == 32'd100)
```

```
    $display("PASS : TC1");
```

```
else
```

```
    $display("FAIL : TC1");
```

```
//-----
```

```
// TC2 : MEMORY DATA SELECT
```

```
//-----
```

```
$display("\nTC2 : Memory Data Select");
```

```
result_src = 2'b01;
```

```
#5;
```

```
assert(data_out == r_data)
```

```
    $display("ASSERT PASS : MEMORY PATH");
```

```
else
```

```
    $error("ASSERT FAIL : MEMORY PATH");
```

```
if(data_out == 32'd200)
```

```
    $display("PASS : TC2");
```

```
else
```

```
    $display("FAIL : TC2");
```

```

//-----
// TC3 : PC+4 SELECT
//-----

$display("\nTC3 : PC+4 Select");

result_src = 2'b10;

#5;

assert(data_out == pc_plus_4_wb)
    $display("ASSERT PASS : PC4 PATH");
else
    $error("ASSERT FAIL : PC4 PATH");

if(data_out == 32'd300)
    $display("PASS : TC3");
else
    $display("FAIL : TC3");

//-----
// TC4 : DEFAULT CASE
//-----

$display("\nTC4 : Default Case");

result_src = 2'b11;

#5;

assert(data_out == alu_result)
    $display("ASSERT PASS : DEFAULT PATH");
else
    $error("ASSERT FAIL : DEFAULT PATH");

```

```
if(data_out == 32'd100)
    $display("PASS : TC4");
else
    $display("FAIL : TC4");

//-----
// TC5 : DYNAMIC SWITCHING
//-----

$display("\nTC5 : Dynamic Switching");

result_src = 2'b00;
#5;

if(data_out != 32'd100)
    $display("FAIL : TC5-1");

result_src = 2'b01;
#5;

if(data_out != 32'd200)
    $display("FAIL : TC5-2");

result_src = 2'b10;
#5;

if(data_out != 32'd300)
    $display("FAIL : TC5-3");

result_src = 2'b11;
#5;

if(data_out != 32'd100)
```

```

        $display("FAIL : TC5-4");

    if(
        data_out == 32'd100
    )
        $display("PASS : TC5");

    //-----

    $display("\nMUX_WB VERIFICATION COMPLETE");

    #10;

    $display("\nCoverage = %0.2f%%",
        cg.get_coverage());

    $finish;

end

final begin
    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());
end

endmodule

```


17.Mux

```
`timescale 1ns/1ps
```

```
module tb_mux;
```

```
// Inputs
```

```
reg [31:0] pc_plus_4;
```

```
reg [31:0] PCTarget;
```

```
reg PCSrc;
```

```
// Output
```

```
wire [31:0] next_pc;
```

```
// DUT
```

```
mux dut(
```

```
    .pc_plus_4(pc_plus_4),
```

```
    .PCTarget(PCTarget),
```

```
    .PCSrc(PCSrc),
```

```
    .next_pc(next_pc)
```

```
);
```

```
//-----
```

```
// Functional Coverage
```

```
//-----
```

```
covergroup mux_cg @(PCSrc);
```

```
    option.per_instance = 1;
```

```
    option.name = "MUX_COVERAGE";
```

```

    cp_pcsrc : coverpoint PCSrc
    {
        bins pc_plus_4_path = {0};
        bins target_path    = {1};
    }

endgroup

mux_cg cg = new();

//-----
// Assertions
//-----

always @(*) begin

    if(PCSrc == 1'b0) begin

        if(next_pc == pc_plus_4)
            $display("ASSERT PASS : PC+4 PATH");
        else
            $error("ASSERT FAIL : PC+4 PATH");

    end

end

always @(*) begin

    if(PCSrc == 1'b1) begin

        if(next_pc == PCTarget)
            $display("ASSERT PASS : TARGET PATH");
        else

```

```

        $error("ASSERT FAIL : TARGET PATH");

    end

end

//-----
// Testcases
//-----

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_mux);

    //-----
    // TC1 : PC+4 PATH
    //-----

    $display("\nTC1 : PC+4 Path");

    pc_plus_4 = 32'd100;
    PCTarget = 32'd500;

    PCSrc = 0;

    #5;

    if(next_pc == 32'd100)
        $display("PASS : TC1");
    else
        $display("FAIL : TC1");

    //-----

```

```
// TC2 : TARGET PATH
```

```
//-----
```

```
$display("\nTC2 : Target Path");
```

```
PCSrc = 1;
```

```
#5;
```

```
if(next_pc == 32'd500)
```

```
    $display("PASS : TC2");
```

```
else
```

```
    $display("FAIL : TC2");
```

```
//-----
```

```
// TC3 : CHANGE INPUTS
```

```
//-----
```

```
$display("\nTC3 : Dynamic Inputs");
```

```
pc_plus_4 = 32'd200;
```

```
PCTarget = 32'd1000;
```

```
PCSrc = 0;
```

```
#5;
```

```
if(next_pc == 32'd200)
```

```
    $display("PASS : TC3-1");
```

```
else
```

```
    $display("FAIL : TC3-1");
```

```
PCSrc = 1;
```

```
#5;
```

```
if(next_pc == 32'd1000)
```

```
    $display("PASS : TC3-2");
```

```
else
```

```
    $display("FAIL : TC3-2");
```

```
//-----
```

```
// TC4 : ZERO VALUES
```

```
//-----
```

```
$display("\nTC4 : Zero Values");
```

```
pc_plus_4 = 0;
```

```
PCTarget = 0;
```

```
PCSrc = 0;
```

```
#5;
```

```
if(next_pc == 0)
```

```
    $display("PASS : TC4");
```

```
else
```

```
    $display("FAIL : TC4");
```

```
//-----
```

```
// TC5 : MAX VALUES
```

```
//-----
```

```
$display("\nTC5 : Max Values");
```

```
pc_plus_4 = 32'hFFFFFFFF;
```

```
PCTarget = 32'hAAAAAAAA;
```

```
PCSrc = 1;
```

```
#5;
```

```
if(next_pc == 32'hAAAAAAAA)
```

```
    $display("PASS : TC5");
```

```
else
```

```
    $display("FAIL : TC5");
```

```
//-----
```

```
$display("\nMUX VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
    $display("\nFinal Coverage = %0.2f%%",
```

```
        cg.get_coverage());
```

```
end
```

```
endmodule
```

18.MuxSrc1

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_muxSrc1;

    reg [31:0] rd1;
    reg [31:0] alu_result;
    reg [31:0] mem_result;
    reg [1:0] forward_A;

    wire [31:0] src1;

    // DUT

    muxSrc1 dut(
        .rd1(rd1),
        .alu_result(alu_result),
        .mem_result(mem_result),
        .forward_A(forward_A),
        .src1(src1)
    );

    //-----
    // COVERAGE
    //-----

    covergroup muxSrc1_cg @(forward_A);

    option.per_instance = 1;
    option.name = "MUXSRC1_COVERAGE";
```

```
cp_forward_A : coverpoint forward_A {
```

```
    bins normal_path = {2'b00};
```

```
    bins alu_forward = {2'b01};
```

```
    bins mem_forward = {2'b10};
```

```
    bins default_path = {2'b11};
```

```
}
```

```
endgroup
```

```
muxSrc1_cg cg = new();
```

```
//-----
```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_muxSrc1);
```

```
    rd1      = 32'd100;
```

```
    alu_result = 32'd200;
```

```
    mem_result = 32'd300;
```

```
//-----
```

```
// TC1 : NORMAL PATH
```

```
//-----
```

```
$display("\nTC1 : NORMAL PATH");
```

```
forward_A = 2'b00;
```



```

#5;

if(src1 == 32'd100) begin
    $display("PASS : TC1");
    $display("ASSERT PASS : NORMAL PATH");
end
else begin
    $display("FAIL : TC1");
    $error("ASSERT FAIL : NORMAL PATH");
end

//-----
// TC2 : ALU FORWARD
//-----

$display("\nTC2 : ALU FORWARD");

forward_A = 2'b01;

#5;

if(src1 == 32'd200) begin
    $display("PASS : TC2");
    $display("ASSERT PASS : ALU FORWARD");
end
else begin
    $display("FAIL : TC2");
    $error("ASSERT FAIL : ALU FORWARD");
end

//-----
// TC3 : MEM FORWARD
//-----

```

```
$display("\nTC3 : MEM FORWARD");
```

```
forward_A = 2'b10;
```

```
#5;
```

```
if(src1 == 32'd300) begin
```

```
    $display("PASS : TC3");
```

```
    $display("ASSERT PASS : MEM FORWARD");
```

```
end
```

```
else begin
```

```
    $display("FAIL : TC3");
```

```
    $error("ASSERT FAIL : MEM FORWARD");
```

```
end
```

```
//-----
```

```
// TC4 : DEFAULT CASE
```

```
//-----
```

```
$display("\nTC4 : DEFAULT PATH");
```

```
forward_A = 2'b11;
```

```
#5;
```

```
if(src1 == 32'd100) begin
```

```
    $display("PASS : TC4");
```

```
    $display("ASSERT PASS : DEFAULT PATH");
```

```
end
```

```
else begin
```

```
    $display("FAIL : TC4");
```

```
    $error("ASSERT FAIL : DEFAULT PATH");
```

```
end
```

```
//-----  
// TC5 : DYNAMIC SWITCHING  
//-----
```

```
$display("\nTC5 : DYNAMIC SWITCHING");
```

```
forward_A = 2'b00;  
#5;
```

```
if(src1 == 32'd100)  
    $display("ASSERT PASS : DYNAMIC NORMAL");  
else  
    $error("ASSERT FAIL : DYNAMIC NORMAL");
```

```
forward_A = 2'b01;  
#5;
```

```
if(src1 == 32'd200)  
    $display("ASSERT PASS : DYNAMIC ALU");  
else  
    $error("ASSERT FAIL : DYNAMIC ALU");
```

```
forward_A = 2'b10;  
#5;
```

```
if(src1 == 32'd300)  
    $display("ASSERT PASS : DYNAMIC MEM");  
else  
    $error("ASSERT FAIL : DYNAMIC MEM");
```

```
forward_A = 2'b11;  
#5;
```

```
if(src1 == 32'd100)
    $display("ASSERT PASS : DYNAMIC DEFAULT");
else
    $error("ASSERT FAIL : DYNAMIC DEFAULT");

$display("PASS : TC5");

//-----

$display("\nMUXSRC1 VERIFICATION COMPLETE");

$display("\nCoverage = %0.2f%%",
    cg.get_coverage());

#10;
$finish;

end

final begin

    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());

end

endmodule
```

19.MuxSrc2

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_muxSrc2;

    reg [31:0] rd2;
    reg [31:0] alu_result;
    reg [31:0] mem_result;
    reg [1:0] forward_B;

    wire [31:0] src2;

    // DUT

    muxSrc2 dut(
        .rd2(rd2),
        .alu_result(alu_result),
        .mem_result(mem_result),
        .forward_B(forward_B),
        .src2(src2)
    );

    //-----
    // COVERAGE
    //-----

    covergroup muxSrc2_cg @(forward_B);

    option.per_instance = 1;
    option.name = "MUXSRC2_COVERAGE";
```

```
cp_forward_B : coverpoint forward_B {
```

```
    bins normal_path = {2'b00};
```

```
    bins alu_forward = {2'b01};
```

```
    bins mem_forward = {2'b10};
```

```
    bins default_path = {2'b11};
```

```
}
```

```
endgroup
```

```
muxSrc2_cg cg = new();
```

```
//-----
```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_muxSrc2);
```

```
    rd2      = 32'd100;
```

```
    alu_result = 32'd200;
```

```
    mem_result = 32'd300;
```

```
//-----
```

```
// TC1 : NORMAL PATH
```

```
//-----
```

```
$display("\nTC1 : NORMAL PATH");
```

```
forward_B = 2'b00;
```

```

#5;

if(src2 == 32'd100) begin
    $display("PASS : TC1");
    $display("ASSERT PASS : NORMAL PATH");
end
else begin
    $display("FAIL : TC1");
    $error("ASSERT FAIL : NORMAL PATH");
end

//-----
// TC2 : ALU FORWARD
//-----

$display("\nTC2 : ALU FORWARD");

forward_B = 2'b01;

#5;

if(src2 == 32'd200) begin
    $display("PASS : TC2");
    $display("ASSERT PASS : ALU FORWARD");
end
else begin
    $display("FAIL : TC2");
    $error("ASSERT FAIL : ALU FORWARD");
end

//-----
// TC3 : MEM FORWARD
//-----

```

```
$display("\nTC3 : MEM FORWARD");
```

```
forward_B = 2'b10;
```

```
#5;
```

```
if(src2 == 32'd300) begin
```

```
    $display("PASS : TC3");
```

```
    $display("ASSERT PASS : MEM FORWARD");
```

```
end
```

```
else begin
```

```
    $display("FAIL : TC3");
```

```
    $error("ASSERT FAIL : MEM FORWARD");
```

```
end
```

```
//-----
```

```
// TC4 : DEFAULT PATH
```

```
//-----
```

```
$display("\nTC4 : DEFAULT PATH");
```

```
forward_B = 2'b11;
```

```
#5;
```

```
if(src2 == 32'd100) begin
```

```
    $display("PASS : TC4");
```

```
    $display("ASSERT PASS : DEFAULT PATH");
```

```
end
```

```
else begin
```

```
    $display("FAIL : TC4");
```

```
    $error("ASSERT FAIL : DEFAULT PATH");
```

```
end
```



```
//-----  
// TC5 : DYNAMIC SWITCHING  
//-----
```

```
$display("\nTC5 : DYNAMIC SWITCHING");
```

```
forward_B = 2'b00;  
#5;
```

```
if(src2 == 32'd100)  
    $display("ASSERT PASS : DYNAMIC NORMAL");  
else  
    $error("ASSERT FAIL : DYNAMIC NORMAL");
```

```
forward_B = 2'b01;  
#5;
```

```
if(src2 == 32'd200)  
    $display("ASSERT PASS : DYNAMIC ALU");  
else  
    $error("ASSERT FAIL : DYNAMIC ALU");
```

```
forward_B = 2'b10;  
#5;
```

```
if(src2 == 32'd300)  
    $display("ASSERT PASS : DYNAMIC MEM");  
else  
    $error("ASSERT FAIL : DYNAMIC MEM");
```

```
forward_B = 2'b11;  
#5;
```

```
if(src2 == 32'd100)
```

```

        $display("ASSERT PASS : DYNAMIC DEFAULT");
    else
        $error("ASSERT FAIL : DYNAMIC DEFAULT");

    $display("PASS : TC5");

    //-----

    $display("\nMUXSRC2 VERIFICATION COMPLETE");

    $display("\nCoverage = %0.2f%%",
        cg.get_coverage());

    #10;
    $finish;

end

final begin

    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());
end

endmodule

```

20.MuxSrcImm

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_muxSrcImm;

    reg alu_src;
    reg [31:0] src2_in;
    reg [31:0] imm;

    wire [31:0] src2;

    // DUT

    muxSrcImm dut(
        .alu_src(alu_src),
        .src2_in(src2_in),
        .imm(imm),
        .src2(src2)
    );

    //-----
    // COVERAGE
    //-----

    covergroup muxSrcImm_cg @(alu_src);

        option.per_instance = 1;
        option.name = "MUXSRCIMM_COVERAGE";

        cp_alu_src : coverpoint alu_src {
```

```

        bins reg_path = {0};
        bins imm_path = {1};

    }

endgroup

muxSrcImm_cg cg = new();

//-----
// TESTCASES
//-----

initial begin

    $dumpfile("dump.vcd");
    $dumpvars(0,tb_muxSrcImm);

    src2_in = 32'd100;
    imm     = 32'd500;

    //-----
    // TC1 : Register Path
    //-----

    $display("\nTC1 : Register Path");

    alu_src = 0;

    #5;

    if(src2 == 32'd100) begin
        $display("PASS : TC1");
        $display("ASSERT PASS : REGISTER PATH");
    end
end

```

```

end
else begin
    $display("FAIL : TC1");
    $error("ASSERT FAIL : REGISTER PATH");
end

//-----
// TC2 : Immediate Path
//-----

$display("\nTC2 : Immediate Path");

alu_src = 1;

#5;

if(src2 == 32'd500) begin
    $display("PASS : TC2");
    $display("ASSERT PASS : IMMEDIATE PATH");
end
else begin
    $display("FAIL : TC2");
    $error("ASSERT FAIL : IMMEDIATE PATH");
end

//-----
// TC3 : Dynamic Switching
//-----

$display("\nTC3 : Dynamic Switching");

alu_src = 0;

#5;

```

```

if(src2 == 32'd100)
    $display("ASSERT PASS : DYNAMIC REGISTER");
else
    $error("ASSERT FAIL : DYNAMIC REGISTER");

alu_src = 1;

#5;

if(src2 == 32'd500)
    $display("ASSERT PASS : DYNAMIC IMMEDIATE");
else
    $error("ASSERT FAIL : DYNAMIC IMMEDIATE");

$display("PASS : TC3");

//-----
// TC4 : Zero Values
//-----

$display("\nTC4 : Zero Values");

src2_in = 0;
imm     = 0;

alu_src = 0;
#5;
if(src2 == 0)
    $display("PASS : TC4");
else
    $display("FAIL : TC4");
//-----
// TC5 : Large Values

```

```
//-----
```

```
$display("\nTC5 : Large Values");
```

```
src2_in = 32'hFFFFFFFF;
```

```
imm     = 32'hAAAAAAAAA;
```

```
alu_src = 1;
```

```
#5;
```

```
if(src2 == 32'hAAAAAAAAA)
```

```
    $display("PASS : TC5");
```

```
else
```

```
    $display("FAIL : TC5");
```

```
//-----
```

```
$display("\nMUXSRCIMM VERIFICATION COMPLETE");
```

```
$display("\nCoverage = %0.2f%%",
```

```
    cg.get_coverage());
```

```
#10;
```

```
$finish;
```

```
end
```

```
final begin
```

```
    $display("\nFinal Coverage = %0.2f%%",
```

```
        cg.get_coverage());
```

```
end
```

```
endmodule
```

21.PC

```
`timescale 1ns/1ps
```

```
module tb_pc;
```

```
    reg clk;
```

```
    reg rst;
```

```
    reg pc_write;
```

```
    reg [31:0] next_pc;
```

```
    wire [31:0] pc;
```

```
    // DUT
```

```
    pc dut(
```

```
        .clk(clk),
```

```
        .rst(rst),
```

```
        .pc_write(pc_write),
```

```
        .next_pc(next_pc),
```

```
        .pc(pc)
```

```
    );
```

```
    // Clock
```

```
    always #5 clk = ~clk;
```

```
    //-----
```

```
    // COVERAGE
```

```
    //-----
```

```
    covergroup pc_cg @(posedge clk);
```



```
option.per_instance = 1;  
option.name = "PC_COVERAGE";
```

```
cp_rst : coverpoint rst {  
    bins reset_on = {1};  
    bins reset_off = {0};  
}
```

```
cp_write : coverpoint pc_write {  
    bins write_enable = {1};  
    bins write_disable = {0};  
}
```

```
endgroup
```

```
pc_cg cg = new();
```

```
//-----  
// SVA ASSERTIONS  
//-----
```

```
property p_reset;  
    @(posedge clk)  
    rst |=> (pc == 32'b0);  
endproperty
```

```
property p_pc_update;  
    @(posedge clk)  
    (!rst && pc_write) |=> (pc == $past(next_pc));  
endproperty
```

```
property p_stall;  
    @(posedge clk)  
    (!rst && !pc_write) |=> (pc == $past(pc));
```

```
endproperty
```

```
assert property(p_reset)
```

```
    $display("SVA PASS : RESET");
```

```
else
```

```
    $error("SVA FAIL : RESET");
```

```
assert property(p_pc_update)
```

```
    $display("SVA PASS : UPDATE");
```

```
else
```

```
    $error("SVA FAIL : UPDATE");
```

```
assert property(p_stall)
```

```
    $display("SVA PASS : STALL");
```

```
else
```

```
    $error("SVA FAIL : STALL");
```

```
//-----
```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_pc);
```

```
    clk = 0;
```

```
//-----
```

```
// TC1 : RESET
```

```
//-----
```

```
    $display("\nTC1 : RESET");
```

```

rst = 1;
pc_write = 0;
next_pc = 0;

#12;

if(pc == 0) begin
    $display("PASS : TC1");
    $display("ASSERT PASS : RESET");
end
else begin
    $display("FAIL : TC1");
    $error("ASSERT FAIL : RESET");
end

//-----
// TC2 : NORMAL UPDATE
//-----

$display("\nTC2 : NORMAL UPDATE");

rst = 0;
pc_write = 1;
next_pc = 32'd100;

#10;

if(pc == 32'd100) begin
    $display("PASS : TC2");
    $display("ASSERT PASS : UPDATE");
end
else begin
    $display("FAIL : TC2");
    $error("ASSERT FAIL : UPDATE");
end

```

end

//-----

// TC3 : STALL

//-----

\$display("\nTC3 : STALL");

pc_write = 0;

next_pc = 32'd500;

#10;

if(pc == 32'd100) begin

 \$display("PASS : TC3");

 \$display("ASSERT PASS : STALL");

end

else begin

 \$display("FAIL : TC3");

 \$error("ASSERT FAIL : STALL");

end

//-----

// TC4 : SECOND UPDATE

//-----

\$display("\nTC4 : SECOND UPDATE");

pc_write = 1;

next_pc = 32'd200;

#10;

if(pc == 32'd200) begin

```
    $display("PASS : TC4");
    $display("ASSERT PASS : SECOND UPDATE");
end
else begin
    $display("FAIL : TC4");
    $error("ASSERT FAIL : SECOND UPDATE");
end
```

```
//-----
// TC5 : LARGE VALUE
//-----
```

```
$display("\nTC5 : LARGE VALUE");
```

```
next_pc = 32'hFFFFFFF0;
```

```
#10;
```

```
if(pc == 32'hFFFFFFF0) begin
    $display("PASS : TC5");
    $display("ASSERT PASS : LARGE VALUE");
end
else begin
    $display("FAIL : TC5");
    $error("ASSERT FAIL : LARGE VALUE");
end
```

```
//-----
// TC6 : RESET AGAIN
//-----
```

```
$display("\nTC6 : RESET AGAIN");
```

```
rst = 1;
```

```

#10;

if(pc == 0) begin
    $display("PASS : TC6");
    $display("ASSERT PASS : RESET AGAIN");
end
else begin
    $display("FAIL : TC6");
    $error("ASSERT FAIL : RESET AGAIN");
end

//-----

$display("\nPC VERIFICATION COMPLETE");

$display("\nCoverage = %0.2f%%",
    cg.get_coverage());

#10;
$finish;

end

final begin

    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());
end

endmodule

```

22. Pc_offset

```
`timescale 1ns/1ps

module tb_pc_offset;

    reg [31:0] pc;
    reg [31:0] imm;

    wire [31:0] pc_offset_out;

    // DUT

    pc_offset dut(
        .pc(pc),
        .imm(imm),
        .pc_offset_out(pc_offset_out)
    );

    //-----
    // COVERAGE
    //-----

    covergroup pc_offset_cg @(pc or imm);

        option.per_instance = 1;
        option.name = "PC_OFFSET_COVERAGE";

        cp_pc : coverpoint pc {

            bins zero_pc  = {0};
            bins small_pc = {[1:100]};
            bins large_pc  = {[101:$]};
```

```
}
```

```
cp_imm : coverpoint imm {
```

```
    bins zero_imm    = {0};
```

```
    bins positive_imm = {[1:1000]};
```

```
    bins negative_imm = {[-1000:-1]};
```

```
    bins large_imm    = {[1001:$]};
```

```
}
```

```
endgroup
```

```
pc_offset_cg cg = new();
```

```
//-----
```

```
// TESTCASES
```

```
//-----
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0,tb_pc_offset);
```

```
//-----
```

```
// TC1 : ZERO + ZERO
```

```
//-----
```

```
    $display("\nTC1 : ZERO + ZERO");
```



```

pc = 0;
imm = 0;

#5;

if(pc_offset_out == 0) begin
    $display("PASS : TC1");
    $display("ASSERT PASS : ZERO");
end
else begin
    $display("FAIL : TC1");
    $error("ASSERT FAIL : ZERO");
end

//-----
// TC2 : POSITIVE OFFSET
//-----

$display("\nTC2 : POSITIVE OFFSET");

pc = 100;
imm = 20;

#5;

if(pc_offset_out == 120) begin
    $display("PASS : TC2");
    $display("ASSERT PASS : POSITIVE OFFSET");
end
else begin
    $display("FAIL : TC2");
    $error("ASSERT FAIL : POSITIVE OFFSET");
end

```

```

//-----
// TC3 : NEGATIVE OFFSET
//-----

$display("\nTC3 : NEGATIVE OFFSET");

pc = 100;
imm = -20;

#5;

if(pc_offset_out == 80) begin
    $display("PASS : TC3");
    $display("ASSERT PASS : NEGATIVE OFFSET");
end
else begin
    $display("FAIL : TC3");
    $error("ASSERT FAIL : NEGATIVE OFFSET");
end

//-----
// TC4 : LARGE VALUES
//-----

$display("\nTC4 : LARGE VALUES");

pc = 32'd100000;
imm = 32'd50000;

#5;

if(pc_offset_out == 32'd150000) begin
    $display("PASS : TC4");
    $display("ASSERT PASS : LARGE VALUES");
end

```

```

end
else begin
    $display("FAIL : TC4");
    $error("ASSERT FAIL : LARGE VALUES");
end

//-----
// TC5 : OVERFLOW
//-----

$display("\nTC5 : OVERFLOW");

pc = 32'hFFFFFFFF;
imm = 32'd1;

#5;

if(pc_offset_out == 32'h00000000) begin
    $display("PASS : TC5");
    $display("ASSERT PASS : OVERFLOW");
end
else begin
    $display("FAIL : TC5");
    $error("ASSERT FAIL : OVERFLOW");
end

//-----
// TC6 : DYNAMIC SWITCHING
//-----

$display("\nTC6 : DYNAMIC SWITCHING");
pc = 200;
imm = 50;
#5;
if(pc_offset_out == 250)
    $display("ASSERT PASS : DYNAMIC-1");

```

```

else
    $error("ASSERT FAIL : DYNAMIC-1");

pc = 300;
imm = -100;

#5;

if(pc_offset_out == 200)
    $display("ASSERT PASS : DYNAMIC-2");
else
    $error("ASSERT FAIL : DYNAMIC-2");

$display("PASS : TC6");

//-----

$display("\nPC_OFFSET VERIFICATION COMPLETE");

$display("\nCoverage = %0.2f%%",
    cg.get_coverage());

#10;
$finish;

end

final begin

    $display("\nFinal Coverage = %0.2f%%",
        cg.get_coverage());
end

endmodule

```

23.Regfile

```
// Code your testbench here
```

```
// or browse Examples
```

```
`timescale 1ns/1ps
```

```
module tb_reg_file;
```

```
reg clk;
```

```
reg rst;
```

```
reg reg_write;
```

```
reg [4:0] rs1;
```

```
reg [4:0] rs2;
```

```
reg [4:0] rd;
```

```
reg [31:0] write_data;
```

```
wire [31:0] read_data1;
```

```
wire [31:0] read_data2;
```

```
// DUT
```

```
reg_file dut (
```

```
    .clk(clk),
```

```
    .rst(rst),
```

```
    .reg_write(reg_write),
```

```
    .rs1(rs1),
```

```
    .rs2(rs2),
```

```
    .rd(rd),
```

```
    .write_data(write_data),
```

```
    .read_data1(read_data1),
```

```
    .read_data2(read_data2)
```

```
);
```

```
//covergroups
```

```
covergroup regfile_cg @(posedge clk);
```

```
    option.per_instance = 1;
```

```
option.name = "REGFILE_COVERAGE";
```

```
cp_rst : coverpoint rst {
```

```
    bins asserted  = {1};
```

```
    bins deasserted = {0};
```

```
}
```

```
cp_reg_write : coverpoint reg_write {
```

```
    bins off = {0};
```

```
    bins on  = {1};
```

```
}
```

```
cp_rd : coverpoint rd {
```

```
    bins x0    = {0};
```

```
    bins low_reg = {[1:10]};
```

```
    bins mid_reg = {[11:20]};
```

```
    bins high_reg = {[21:31]};
```

```
}
```

```
cp_rs1 : coverpoint rs1 {
```

```
    bins x0    = {0};
```

```
    bins low_reg = {[1:10]};
```

```
bins mid_reg = {[11:20]};
```

```
bins high_reg = {[21:31]};  
}
```

```
cp_rs2 : coverpoint rs2 {
```

```
bins x0      = {0};
```

```
bins low_reg = {[1:10]};
```

```
bins mid_reg = {[11:20]};
```

```
bins high_reg = {[21:31]};  
}
```

```
cp_data : coverpoint write_data {
```

```
bins data_low = {[0:255]};
```

```
bins data_mid = {[256:1023]};
```

```
bins data_high = {[1024:$]};
```

```
}
```

```
cross_write_rd :
```

```
cross cp_reg_write, cp_rd;
```

```
cross_rst_write :
```

```
cross cp_rst, cp_reg_write;
```

```
endgroup
```

```
regfile_cg cg = new();
```

```
// Clock Generation
```

```
always #5 clk = ~clk;
```

```
//ASSERTIONS
```

```
//Assertion 1: x0 must always remain zero
```

```
property p_x0_zero;
```

```
@(posedge clk)
```

```
disable iff(rst)
```

```
dut.register[0] == 32'b0;
```

```
endproperty
```

```
a_x0_zero: assert property(p_x0_zero)
```

```
$display("PASS : x0 remains zero");
```

```
else
```

```
$error("FAIL : x0 modified");
```

```
//Assertion 2: Valid write updates register
```

```
property p_write_success;
```

```
@(posedge clk)
```

```
disable iff(rst)
```

```
(reg_write && rd!=0)
```

```
==> (dut.register[$past(rd)] == $past(write_data));
```

```
endproperty
```

```
a_write_success: assert property(p_write_success)
```

```
$display("PASS : Register Write");
```



```

else
    $error("FAIL : Register Write");

    //Assertion 3: Write to x0 ignored

property p_x0_protection;
    @(posedge clk)
    (reg_write && rd==0)
    |=> (dut.register[0] == 0);
endproperty

a_x0_protection: assert property(p_x0_protection)
    $display("PASS : x0 Protection");
else
    $error("FAIL : x0 Protection");

    //Assertion 4: Reset clears registers

property p_reset_clear;
    @(posedge clk)
    $rose(rst)
    |=> (
        dut.register[1]==0 &&
        dut.register[5]==0 &&
        dut.register[10]==0
    );
endproperty

a_reset_clear: assert property(p_reset_clear)
    $display("PASS : Reset Clear");
else
    $error("FAIL : Reset Clear");

    //Assertion 5: No write when reg_write=0

```

```
// property p_no_write;
// @(posedge clk)
// (!reg_write)
// | => (dut.register[rd] == $past(dut.register[rd]));
// endproperty
```

```
// a_no_write: assert property(p_no_write)
// $display("PASS : No Write");
// else
// $error("FAIL : Unexpected Write");
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
    $dumpvars(0,tb_reg_file);
```

```
    clk = 0;
    rst = 0;
    reg_write = 0;
```

```
    rs1 = 0;
    rs2 = 0;
    rd = 0;
    write_data = 0;
```

```
//-----
// TC1 : RESET VERIFICATION
//-----
```

```
$display("\nTC1 : Reset Verification");
```

```
    rst = 1;
```

```
#12;
```

```
rst = 0;
```

```
rs1 = 5;
```

```
#2;
```

```
if(read_data1 == 0)
```

```
    $display("PASS : TC1");
```

```
else
```

```
    $display("FAIL : TC1");
```

```
//-----
```

```
// TC2 : SINGLE REGISTER WRITE
```

```
//-----
```

```
$display("\nTC2 : Single Register Write");
```

```
reg_write = 1;
```

```
rd = 5;
```

```
write_data = 32'd100;
```

```
#10;
```

```
reg_write = 0;
```

```
rs1 = 5;
```

```
#2;
```

```
if(read_data1 == 32'd100)
```

```
    $display("PASS : TC2");
```

```

else
    $display("FAIL : TC2");

//-----
// TC3 : DUAL READ VERIFICATION
//-----

$display("\nTC3 : Dual Read Verification");

reg_write = 1;

rd = 10;
write_data = 32'd200;

#10;

reg_write = 0;

rs1 = 5;
rs2 = 10;

#2;

if(read_data1 == 32'd100 &&
    read_data2 == 32'd200)
    $display("PASS : TC3");
else
    $display("FAIL : TC3");

//-----
// TC4 : WRITE DISABLE
//-----

$display("\nTC4 : Write Disable Verification");

```

```
reg_write = 0;
```

```
rd = 6;
```

```
write_data = 32'd999;
```

```
#10;
```

```
rs1 = 6;
```

```
#2;
```

```
if(read_data1 == 0)
```

```
    $display("PASS : TC4");
```

```
else
```

```
    $display("FAIL : TC4");
```

```
//-----
```

```
// TC5 : x0 PROTECTION
```

```
//-----
```

```
$display("\nTC5 : x0 Protection");
```

```
reg_write = 1;
```

```
rd = 0;
```

```
write_data = 32'd12345;
```

```
#10;
```

```
reg_write = 0;
```

```
rs1 = 0;
```

```

#2;

if(read_data1 == 0)
    $display("PASS : TC5");
else
    $display("FAIL : TC5");

//-----
// TC6 : MULTIPLE REGISTER WRITES
//-----

$display("\nTC6 : Multiple Register Writes");

reg_write = 1;

rd = 1;
write_data = 32'd111;
#10;

rd = 2;
write_data = 32'd222;
#10;

rd = 3;
write_data = 32'd333;
#10;

reg_write = 0;

rs1 = 1;
#2;
if(read_data1 == 111)
    $display("PASS : x1");

```

```

rs1 = 2;
#2;
if(read_data1 == 222)
    $display("PASS : x2");

rs1 = 3;
#2;
if(read_data1 == 333)
    $display("PASS : x3");

//-----
// TC7 : REGISTER UPDATE
//-----

$display("\nTC7 : Register Update");

reg_write = 1;

rd = 8;
write_data = 32'd100;

#10;

rs1 = 8;

#2;

if(read_data1 == 100)
    $display("PASS : Update Step 1");
else
    $display("FAIL : Update Step 1");

write_data = 32'd500;

```

```
#10;
```

```
reg_write = 0;
```

```
rs1 = 8;
```

```
#2;
```

```
if(read_data1 == 500)
```

```
    $display("PASS : Update Step 2");
```

```
else
```

```
    $display("FAIL : Update Step 2");
```

```
//-----
```

```
// TC8 : Coverage Closure
```

```
//-----
```

```
$display("\nTC8 : Coverage Closure");
```

```
// Cover rd mid/high bins
```

```
reg_write = 1;
```

```
rd = 15;
```

```
write_data = 32'd555;
```

```
#10;
```

```
rd = 25;
```

```
write_data = 32'd777;
```

```
#10;
```

```
// Cover rs1 bins
```

```
rs1 = 15;
```



```
#2;
```

```
rs1 = 25;
```

```
#2;
```

```
// Cover rs2 bins
```

```
rs2 = 16;
```

```
#2;
```

```
rs2 = 26;
```

```
#2;
```

```
// Cover cross_write_rd bins
```

```
reg_write = 0;
```

```
rd = 15;
```

```
#10;
```

```
rd = 25;
```

```
#10;
```

```
// Cover cross_rst_write
```

```
rst = 1;
```

```
reg_write = 1;
```

```
#10;
```

```
rst = 0;
```

```
reg_write = 0;
```

```
//-----
```

```
// TC9 : Mid Register Access

//-----

$display("\nTC9 : Mid Register Coverage");

// write register 15

reg_write = 1;
rd = 15;
write_data = 32'd1500;

#10;

reg_write = 0;

// read through rs1

rs1 = 15;

#2;

if(read_data1 == 32'd1500)
    $display("PASS : rs1 mid_reg");

// read through rs2

rs2 = 15;

#2;

if(read_data2 == 32'd1500)
    $display("PASS : rs2 mid_reg");

//-----
```

```
$display("\nREG_FILE VERIFICATION COMPLETE");
```

```
#20;
```

```
$display("\nCoverage = %0.2f%%",  
        cg.get_coverage());
```

```
$finish;
```

```
end
```

```
final begin
```

```
$display("\nFinal Coverage = %0.2f%%",  
        cg.get_coverage());
```

```
end
```

```
endmodule
```

24.Risc_top

// Code your testbench here

// or browse Examples

`timescale 1ns/1ps

module tb_risc_top;

//-----

// SIGNALS

//-----

reg clk;

reg rst;

//-----

// DUT

//-----

risc_top dut (

 .clk(clk),

 .rst(rst)

);

//-----

// COVERAGE

//-----

covergroup risc_top_cg @(posedge clk);

option.per_instance = 1;

option.name = "RISC_TOP_COVERAGE";

// Branch taken/not taken

```
cp_pcsrc : coverpoint dut.pc_src {  
    bins not_taken = {0};  
    bins taken     = {1};  
}
```

```
// Flush coverage
```

```
cp_flush : coverpoint dut.flush {  
    bins no_flush = {0};  
    bins flush    = {1};  
}
```

```
// Hazard coverage
```

```
cp_stall : coverpoint dut.control_mux_sel_id {  
    bins normal = {0};  
    bins stall  = {1};  
}
```

```
// Forwarding coverage
```

```
cp_forwardA : coverpoint dut.forwardA {  
    bins no_forward = {2'b00};  
    bins ex_forward = {2'b01};  
    bins mem_forward = {2'b10};  
}
```

```
cp_forwardB : coverpoint dut.forwardB {  
    bins no_forward = {2'b00};  
    bins ex_forward = {2'b01};  
    bins mem_forward = {2'b10};  
}
```

```
// WB coverage
```

```
cp_regwrite : coverpoint dut.reg_write_wb {  
    bins write_off = {0};  
    bins write_on  = {1};  
}
```

```
// Memory coverage
```

```
cp_memread : coverpoint dut.mem_read_mem {  
    bins no_read = {0};  
    bins read    = {1};  
}
```

```
cp_memwrite : coverpoint dut.mem_write_mem {  
    bins no_write = {0};  
    bins write    = {1};  
}
```

```
endgroup
```

```
risc_top_cg cg = new();
```

```
//-----
```

```
// CLOCK
```

```
//-----
```

```
initial begin
```

```
    clk = 0;
```

```
    forever #5 clk = ~clk;
```

```
end
```

```
//ASSERTIONS
```

```
property p_reset_pc;  
@(posedge clk)  
rst |=> (dut.pc == 0);  
endproperty
```

```
assert property(p_reset_pc)  
$display("ASSERT PASS : RESET PC");  
else  
$error("ASSERT FAIL : RESET PC");
```

```
property p_pc_update;  
@(posedge clk)  
disable iff(rst)  
(dut.pc_write && !$past(rst))  
|=> (dut.pc == $past(dut.next_pc));  
endproperty
```

```
assert property(p_pc_update)  
$display("ASSERT PASS : PC UPDATE");  
else  
$error("ASSERT FAIL : PC UPDATE");
```

```
property p_pc_stall;  
@(posedge clk)  
disable iff(rst)
```

```
(!dut.pc_write && dut.control_mux_sel_id)
```

```
|=>
```

```
(dut.pc == $past(dut.pc));
```

```
endproperty
```

```
assert property(p_pc_stall)
```

```
    $display("ASSERT PASS : STALL");
```

```
else
```

```
    $error("ASSERT FAIL : STALL");
```

```
property p_branch_taken;
```

```
    @(posedge clk)
```

```
    disable iff(rst)
```

```
    dut.pc_src |-> dut.flush;
```

```
endproperty
```

```
assert property(p_branch_taken)
```

```
    $display("ASSERT PASS : BRANCH FLUSH");
```

```
else
```

```
    $error("ASSERT FAIL : BRANCH FLUSH");
```

```
property p_stall;
```

```
    @(posedge clk)
```

```
    dut.control_mux_sel_id
```

```
    |->
```

```
    (dut.pc_write == 0);
```

```
endproperty
```

```
assert property(p_stall)
```

```
    $display("ASSERT PASS : STALL");
```

```
else
```

```
    $error("ASSERT FAIL : STALL");
```



```
property p_branch_flush;
@(posedge clk)
dut.pc_src |-> dut.flush;
endproperty
```

```
assert property(p_branch_flush)
$display("ASSERT PASS : BRANCH FLUSH");
else
$error("ASSERT FAIL : BRANCH FLUSH");
```

```
property p_ex_forwardA;
@(posedge clk)
(dut.forwardA == 2'b01)
|->
(dut.reg_write_mem);
endproperty
```

```
assert property(p_ex_forwardA)
$display("ASSERT PASS : EX FORWARD A");
else
$error("ASSERT FAIL : EX FORWARD A");
```

```
property p_mem_forwardA;
@(posedge clk)
(dut.forwardA == 2'b10)
|->
(dut.reg_write_wb);
endproperty
```

```
assert property(p_mem_forwardA)
$display("ASSERT PASS : MEM FORWARD A");
else
```

```
$error("ASSERT FAIL : MEM FORWARD A");
```

```
property writeback_check;  
@(posedge clk)  
disable iff(rst)  
dut.reg_write_wb  
|-> (dut.rd_wb inside {[0:31]});  
endproperty
```

```
assert property(writeback_check)  
$display("ASSERT PASS : WRITEBACK");  
else  
$error("ASSERT FAIL : WRITEBACK");
```

```
property p_memread;  
@(posedge clk)  
dut.mem_read_mem  
|->  
(!dut.mem_write_mem);  
endproperty
```

```
assert property(p_memread)  
$display("ASSERT PASS : MEM READ");  
else  
$error("ASSERT FAIL : MEM READ");
```

```
//-----  
// TESTING  
//-----
```

```
initial begin
```

```

$dumpfile("risc_top.vcd");
$dumpvars(0,tb_risc_top);

//-----
// TC1 : RESET
//-----

$display("\n=====");
$display("TC1 : RESET");
$display("=====");

rst = 1;

repeat(3) @(posedge clk);

rst = 0;

repeat(2) @(posedge clk);

$display("PC = %0d", dut.pc);

//-----
// TC2 : INITIAL FETCH
//-----

$display("\n=====");
$display("TC2 : INSTRUCTION FETCH");
$display("=====");

repeat(5) begin
    @(posedge clk);

    $display(
        "TIME=%0t PC=%0h INSTR=%0h",

```

```

        $time,
        dut.pc,
        dut.instruction_code_if
    );
end

//-----
// TC3 : PIPELINE FLOW
//-----

$display("\n=====");
$display("TC3 : PIPELINE FLOW");
$display("=====");

repeat(10) begin
    @(posedge clk);

    $display(
        "PC=%h IF=%h ID=%h EX_RD=%0d MEM_RD=%0d WB_RD=%0d",
        dut.pc,
        dut.instruction_code_if,
        dut.instruction_code_id,
        dut.rd_ex,
        dut.rd_mem,
        dut.rd_wb
    );
end

//-----
// TC4 : REGISTER WRITEBACK
//-----

$display("\n=====");
$display("TC4 : WRITEBACK");

```

```

$display("=====");

repeat(15) begin
    @(posedge clk);

    if(dut.reg_write_wb)
    begin
        $display(
            "WB -> rd=%0d data=%0h",
            dut.rd_wb,
            dut.write_data_wb
        );
    end
end

//-----
// TC5 : MEMORY ACCESS
//-----

$display("\n=====");
$display("TC5 : MEMORY");
$display("=====");

repeat(15) begin
    @(posedge clk);

    if(dut.mem_read_mem)
    $display(
        "LOAD addr=%0h data=%0h",
        dut.alu_result_mem,
        dut.mem_result_mem
    );

    if(dut.mem_write_mem)

```

```

    $display(
        "STORE addr=%h data=%h",
        dut.alu_result_mem,
        dut.write_data_mem
    );
end

//-----
// TC6 : HAZARD DETECTION
//-----

$display("\n=====");
$display("TC6 : HAZARD CHECK");
$display("=====");

repeat(20) begin
    @(posedge clk);

    if(dut.control_mux_sel_id)
        $display(
            "STALL DETECTED at time=%0t",
            $time
        );
end

//-----
// TC7 : FORWARDING CHECK
//-----

$display("\n=====");
$display("TC7 : FORWARDING CHECK");
$display("=====");

repeat(20) begin

```

```

@(posedge clk);

if(dut.forwardA != 2'b00 ||
   dut.forwardB != 2'b00)
begin
    $display(
        "FORWARDING A=%b B=%b",
        dut.forwardA,
        dut.forwardB
    );
end
end

//-----
// TC8 : BRANCH/JUMP CHECK
//-----

$display("\n=====");
$display("TC8 : BRANCH/JUMP");
$display("=====");

repeat(20) begin
    @(posedge clk);

    if(dut.pc_src)
    begin
        $display(
            "BRANCH/JUMP TAKEN PC=%h TARGET=%h",
            dut.pc,
            dut.pc_target
        );
    end
end
end

```

```
//-----
```

```
// END
```

```
//-----
```

```
$display("\n=====");
```

```
$display("RISC_TOP VERIFICATION COMPLETE");
```

```
$display("=====");
```

```
#20;
```

```
$finish;
```

```
end
```

```
final begin
```

```
    $display("\nFinal Coverage = %0.2f%%",
```

```
        cg.get_coverage());
```

```
end
```

```
endmodule
```


25. Run.do (to run in aldec riveria pro 2025.04)

```
vsim +access+r
```

```
run -all
```

```
acdb save
```

```
acdb report -db fcover.acdb -txt -o cov.txt -verbose
```

```
exec cat cov.txt
```

```
exit
```